

Build Your Own Project – VITyarthi

CreaseIQ: IPL Match Intelligence Platform

A validated data pipeline, honest analytics and a leakage-safe win-probability engine

Course	Machine Learning (Fundamentals of AI & ML)
Student	Jyotshna Payasi
Registration number	26BCE10651
Section	—
University	Vellore Institute of Technology, Bhopal
Date	29 September 2026
Repository	(link added when the repository is published on GitHub)

All figures and numbers in this report are generated from the project's own outputs (`reports/*.json`) by `creaseiq report` .

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

2.1 Context

The Indian Premier League (IPL) is the world's most-watched T20 cricket competition. Since 2008 it has produced 1,243 matches over 19 seasons, and every one has a public scorecard. Fans, journalists and analysts argue about the same questions each season:

- Does winning the toss decide the match?
- Is chasing easier?
- Did the 2023 *Impact Player* rule change the game?
- Can anyone predict who will win?

The data needed to answer them exists, but it is messy. Answers that are both statistically sound and honest about their uncertainty are rare.

2.2 What CreaselQ is

CreaselQ is a complete machine-learning project built on the supplied IPL match file. It has five functional modules:

1. **Data engineering (M1).** Validates, cleans and canonicalises the raw file into a normalised relational database, and records every correction in an auditable quality report.
2. **Analytics & visualisation (M2).** Answers the questions above with proper statistical tests and confidence intervals, through an interactive dashboard.
3. **Prediction engine (M3).** Estimates pre-match win probabilities with *leakage-safe* features, time-ordered validation, calibration and baselines, and explains each prediction.
4. **Scenarios (M4).** What-if analysis (toss, venue, opponent) and a Monte Carlo season simulation.
5. **Operations & reporting (M5).** Structured logging, a prediction log, drift monitoring and generated evidence (model card, figures, NFR verification).

2.3 Objectives

1. Ingest, validate, clean and canonicalise the dataset, with an auditable quality report.
2. Store it in a normalised schema with reproducible, idempotent rebuilds.
3. Provide statistically sound analytics (toss, venue, team and era effects) that report their uncertainty.
4. Predict match outcomes with leakage-safe features, walk-forward validation, calibrated probabilities and explanations.
5. Offer an accessible dashboard and CLI with logging, monitoring and thorough test coverage.
6. Document the design and results against the assignment's artefact list.

2.4 Relevance to the course

The project is an end-to-end application of machine learning. It covers:

- **Data preprocessing:** schema validation, anomaly correction, imputation.
- **Exploratory and inferential statistics.**
- **Feature engineering:** including sequential Elo ratings.

- **Supervised classification and regression:** logistic regression, random forests, gradient boosting and ridge regression.
- **Unsupervised learning:** k-means venue clustering.
- **Model selection and evaluation for time-ordered data:** walk-forward CV, the one-standard-error rule, bootstrap confidence intervals and paired tests.
- **Probability calibration and interpretability.**
- **ML ethics:** data leakage, honest reporting, a betting disclaimer.

The AI-fundamentals topics of the VITyarthi course are also mapped explicitly. Hyperparameter search is treated as heuristic search, the YAML identity rules as a knowledge base, and the prediction service as a simple agent. `docs/course_mapping.md` maps each topic to its implementing file and test.

2.5 Structure of this report

Sections 3–5 define the problem and the functional and non-functional requirements. Sections 6–8 present the architecture, the design diagrams and the reasoning behind each design decision. Section 9 details the implementation: dataset, cleaning, features, model selection and evaluation methodology. Section 10 shows the results and real screenshots, and Section 11 the testing approach. Sections 12–14 reflect on challenges, learnings and future work, and Section 15 lists the references.

Headline result. CreaseIQ's most important finding is honest: pre-match IPL outcomes are close to a coin flip. The best walk-forward log-loss is 0.6857, against 0.6931 for a coin. On the 2025–26 holdout, which was evaluated only after the selection was frozen, no model beat the coin. Online "predictors" that claim 80–90% accuracy are almost always leaking information (Section 8).

3. Problem Statement

3.1 The problem

IPL data is rich but inconsistent. Several failure modes make naive analysis wrong:

- **Renamed and replaced franchises.** For example, Delhi Daredevils became Delhi Capitals. Deccan Chargers and Sunrisers Hyderabad, however, are *different* franchises.
- **One stadium, many spellings.** A stadium can appear under several spellings or under old and new names.
- **Season labels that are not years.**
- **Hidden semantic traps** that the raw columns never announce.

Students, analysts and fans lack a clean, validated, reproducible dataset. They also lack a principled way to quantify the toss, venue, form and era effects. On top of that, many machine-learning "IPL predictors" published online leak post-match information or ignore time order. They report accuracies that cannot be reproduced on genuinely unseen matches.

CreaseIQ therefore sets out to deliver three things:

1. A validated, canonical dataset with an auditable record of every change.
2. Statistically sound analytics that report uncertainty.
3. A leakage-safe, calibrated pre-match win-probability engine whose accuracy is measured honestly.

3.2 Data traps discovered (and handled)

Table 1: Data problems found during profiling and research, and how they are handled

Trap	Evidence	Handling
<code>team1</code> means "batting first" only from 2018	share of matches with raw <code>team1</code> batting first: 1.00 every season from 2018; 0.45–0.62 before	Batting order is derived from the toss; orientation uses a hash-seeded coin flip (ADR-005)
Tie rows include super-over runs and wickets	16 ties; e.g. a side "losing 12 wickets" (10 + 2 in the super over)	Regulation scores restored from verified external data (ADR-004)
Voided and replayed match stored as "no result"	1 match (08-05-2025, security blackout)	<code>voided</code> flag; excluded from scoring statistics
D/L-affected results	14 flagged by margin logic, 16 from a verified list	<code>dls_flag</code> = heuristic OR list (19 rows)
19 team strings for 15 franchises	renames and spelling variants	Lineage map with validity years (ADR-003)
60 venue strings for 37 venues	suffixes, renames, rebuilt stadiums	Canonical venue map, fail-loud
City "Unknown" in 51 rows	Dubai and Sharjah venues	Imputed from the canonical venue

3.3 Scope

In scope:

- The supplied match file (2008–2026) and validated uploads of new matches.
- A relational database.
- Analytics: toss, chasing, teams, venues, scoring eras, player-of-the-match.

- Two-tier win probability: pre-toss and post-toss.
- First-innings score regression.
- What-if scenarios and a season simulation.
- A dashboard, CLI, tests, CI and documentation.

Out of scope:

- Ball-by-ball or live in-play prediction.
- Betting odds or betting advice.
- Player batting and bowling statistics, which are not in the data.
- Multi-user accounts. CreaseIQ is a single-user local application holding only public match records, so authentication would add risk without benefit.

3.4 Target users

Table 2: Target users and their needs

User	Need	Served by
Students / analysts	Clean data; honest statistics with confidence intervals	Data Explorer, Team Analytics, Venue & Toss, CSV export
Cricket fans	Evidence-based previews; "what if we win the toss?"	Predict Match, What-If
Maintainers / data engineers	Add seasons and sources safely; retrain reproducibly	CLI (<code>ingest</code> , <code>all</code> , <code>train</code>), tests, CI, ADRs

4. Functional Requirements

The assignment requires at least three major functional modules. CreaseIQ has five (M1–M5), with 22 functional requirements. Their IDs are cited in code docstrings and tests, and the full trace is in `docs/rubric_traceability.md`.

Table 3: Functional requirements with priority, implementation and verifying tests

ID	Module	Requirement	Pri.	Implementation	Test
FR-01	M1	Ingest CSV; schema and type enforcement; quarantine bad rows with reasons	P0	<code>data/source.py</code> , <code>ingest.py</code> , <code>schema.py</code>	<code>test_ingest.py</code>
FR-02	M1	Clean and canonicalise teams, venues, cities, seasons; derive batting order, stage, champion, D/L, era	P0	<code>data/cleaning.py</code> , <code>canonical.py</code>	<code>test_cleaning.py</code> , <code>test_canonical.py</code>
FR-03	M1	Data-quality report (MD + JSON) and data dictionary	P0	<code>data/quality_report.py</code>	<code>test_quality_report.py</code>
FR-04	M1	Normalised SQLite store and Parquet cache; idempotent rebuild	P0	<code>db/models.py</code> , <code>loader.py</code> , <code>repository.py</code>	<code>test_db.py</code>
FR-05	M1	Validated CSV append (dry run, dedup, size and type limits)	P1	<code>services/ingest_service.py</code>	<code>test_services.py</code>
FR-06	M2	Team records, titles, season form, head-to-head, rating timeline	P0	<code>analytics/team_stats.py</code>	<code>test_analytics.py</code>
FR-07	M2	Venue analytics with shrinkage; toss habits; clustering	P0	<code>analytics/venue_stats.py</code> , <code>venue_clusters.py</code>	<code>test_analytics.py</code>
FR-08	M2	Toss analysis with hypothesis tests and CIs; era comparison	P0	<code>analytics/toss_analysis.py</code> , <code>hypothesis_tests.py</code>	<code>test_hypothesis_tests.py</code>
FR-09	M2	Scoring and margin trends; closest finishes	P1	<code>analytics/season_trends.py</code>	<code>test_analytics.py</code>
FR-10	M2	Player-of-the-match leaderboard; appearances; squad continuity	P1	<code>analytics/player_stats.py</code>	<code>test_analytics.py</code>
FR-11	M2	Interactive filters and sanitised CSV export	P0	<code>services/analytics_service.py</code> , app page 1	<code>test_services.py</code> , <code>test_app.py</code>

ID	Module	Requirement	Pri.	Implementation	Test
FR-12	M3	Leakage-safe feature builder with an <code>as_of</code> contract	P0	<code>features/builder.py</code> (+ 4 state modules)	<code>test_leakage.py</code>
FR-13	M3	Margin-aware Elo with season carry-over and history	P0	<code>features/elo.py</code> , <code>models/elo_tuning.py</code>	<code>test_features.py</code>
FR-14	M3	≥ 4 model families and ≥ 4 baselines under walk-forward validation	P0	<code>models/train.py</code> , <code>baselines.py</code> , <code>splits.py</code>	<code>test_models.py</code>
FR-15	M3	Calibration and evaluation report with CIs and a reliability diagram	P0	<code>models/evaluate.py</code> , <code>calibrate.py</code>	<code>test_models.py</code>
FR-16	M3	Predict via CLI, API and UI; two tiers; explanation	P0	<code>models/predict.py</code> , <code>explain.py</code> , <code>services/prediction_service.py</code>	<code>test_experiment.py</code> , <code>test_services.py</code>
FR-17	M3	First-innings score regressor vs baselines	P1	<code>models/score_regressor.py</code>	<code>test_experiment.py</code>
FR-18	M3	Model registry with SHA-256, params, data hash and commit	P0	<code>models/registry.py</code>	<code>test_models.py</code>
FR-19	M4	What-if: toss, venue, opponent $\rightarrow \Delta$ probability	P1	<code>simulation/what_if.py</code> , <code>services/scenario_service.py</code>	<code>test_services.py</code>
FR-20	M4	Monte Carlo season simulation	P2	<code>simulation/season_monte_carlo.py</code>	<code>test_services.py</code>
FR-21	M5	Structured logging, prediction log, PSI drift monitor	P0	<code>logging_setup.py</code> , <code>db_prediction_log</code> , <code>reporting/drift.py</code>	<code>test_core.py</code> , <code>test_figures_logging.py</code>
FR-22	M5	Generated model card and report assets	P0	<code>reporting/model_card.py</code> , <code>figures.py</code> , <code>assets.py</code>	<code>test_experiment.py</code>

4.1 Input / output structure of each module

Table 4: Module inputs and outputs

Module	Input	Output
M1 Data engineering	<code>data/raw/ipl_matches.csv</code> (+ appended uploads), <code>configs/*.yaml</code> , <code>data/external/*.csv</code>	<code>matches.parquet</code> , <code>match_players.parquet</code> , SQLite DB, <code>quarantine.csv</code> , quality report, data dictionary
M2 Analytics	Canonical match table, filter values	DataFrames, Plotly charts, <code>reports/analytics.json</code> , <code>docs/analytics_findings.md</code> , safe CSV

Module	Input	Output
M3 Prediction	Canonical matches + squads; config (seeds, grids, splits)	reports/metrics.json , registered models, figures, model card; at serving time: probability, drivers, log row
M4 Scenarios	A fixture (teams, venue, date, optional toss)	What-if table (Δ probability), title/top-4 odds
M5 Ops & reporting	Pipeline events, predictions, feature frame	Log file, prediction_log table, PSI table, NFR evidence, this report

4.2 User workflow

A user opens the dashboard, sees KPIs and data and model status on Home, and then follows one of three paths (workflow diagram D3, Section 7.2):

- **Explore:** filter, inspect and export.
- **Predict:** choose two teams and a venue (all preselected), optionally add the toss, and click *Predict*. That is at most three clicks.
- **Maintain:** upload, dry run, then apply.

5. Non-functional Requirements

The assignment requires at least four non-functional requirements. CreaselQ specifies eight. Each has a measurable target and a verification method, and each is verified with evidence produced by the code (`docs/nfr_verification.md`).

Table 5: Non-functional requirements: targets and verification method

ID	Category	Requirement (target)	Verified by
NFR-01	Performance	Full pipeline ≤ 90 s; single warm prediction ≤ 200 ms; dashboard page ≤ 2 s	<code>creaseiq benchmark</code> $\rightarrow$ <code>reports/perf.json</code> ; <code>test_performance.py</code>
NFR-02	Reliability & error handling	Typed exception hierarchy; idempotent pipeline; friendly errors with no stack traces	Unit, CLI and AppTest tests with bad inputs
NFR-03	Security	Allow-list validation; parameterised SQL only; hash-verified model loading; CSV-only uploads ≤ 5 MB; CSV-injection escaping; no secrets	<code>bandit</code> , <code>pip-audit</code> , <code>test_security.py</code>
NFR-04	Usability	≤ 3 clicks to a prediction; colour-blind-safe palettes; tooltips; empty states; disclaimer	AppTest, screenshots
NFR-05	Maintainability	Type hints, docstrings, ruff and mypy clean, enforced layering, $\geq 85\%$ coverage on core packages	CI, <code>coverage.json</code> , <code>test_architecture.py</code>
NFR-06	Reproducibility	Seeds, pinned dependencies, config-driven; identical metrics across runs	<code>test_experiment.py::test_reproducible_metrics</code>
NFR-07	Logging & monitoring	Structured logs with run id and timings; prediction log; PSI drift panel	<code>test_figures_logging.py</code> , Model Lab
NFR-08	Scalability & efficiency	DB backend by URL; <code>MatchSource</code> interface; $O(n)$ incremental features; RAM < 1 GB	<code>perf.json</code> , interface tests

5.1 Measured results

Table 6: Measured NFR results (generated from evidence files)

ID	Category	Measured	Result
NFR-01	Performance	pipeline ingest $\rightarrow$ DB $\rightarrow$ features $\rightarrow$ train of the registered configuration 6.6 s (≤ 90 ; the one-off model-selection search is reported separately in <code>metrics.json</code>); prediction 9.3 ms median, p95 10.9 ms (≤ 200); slowest page 0.57 s (≤ 2)	PASS
NFR-02	Reliability & error handling	typed exception hierarchy; CLI exit code 1 with a friendly message; UI guard with no stack traces; idempotent DB rebuild (checksum test)	PASS
NFR-03	Security	bandit issues: 0; known vulnerable dependencies: 0; parameterised SQL, hash-verified artifacts, upload guards and CSV-injection escaping all tested	PASS
NFR-04	Usability	prediction in ≤ 3 clicks (defaults preselected, one Predict button); colour-blind-safe palettes; tooltips, empty states and a disclaimer on every page	PASS

ID	Category	Measured	Result
NFR-05	Maintainability	coverage total 94.6%, core minimum 98.7% (data 99%, features 100%, models 99%, analytics 100%); ruff, mypy and the layer contract are clean	PASS
NFR-06	Reproducibility	seeded, config-driven, pinned dependencies; two full experiment runs give identical metrics	PASS
NFR-07	Logging & monitoring	key=value logs with a run_id and per-stage durations (console plus rotating file); prediction_log table; PSI drift report	PASS
NFR-08	Scalability & resource efficiency	peak Python memory 40 MB (< 1024); DB backend chosen by URL; MatchSource protocol; incremental O(n) features; Parquet cache	PASS

Measured on Windows-11-10.0.26200-SP0 (16 logical CPUs, Python 3.12.10).

Pipeline stage times (seconds):

Stage	Time
Data	2.33
Database	2.70
Features	1.47
Training the registered configuration	0.09

NFR-01 covers the operational pipeline, which trains the registered configuration. The one-off model-selection experiment is not part of it. That experiment (the Elo grid, five model families and their grids, 13 walk-forward folds, two tiers, calibration, ablations and bootstrap tests) took 106.5 s in the recorded `creaseiq train` run.

The cross-platform requirement is verified locally on Windows 11, including a clean-clone install and run. The GitHub Actions workflow is configured to run on Ubuntu, Windows and macOS with Python 3.12 and 3.13, and it executes on every push once the repository is published.

6. System Architecture

CreaselQ uses a **layered architecture in which dependencies point downward only**. The presentation layer (Streamlit dashboard and Typer CLI) calls a services layer. Services orchestrate the domain packages: analytics, features, models, simulation, reporting and visualisation. The domain packages sit on a data-access layer (sources, validation, cleaning, ORM, repositories), which in turn uses a small core of configuration, logging and exception utilities.

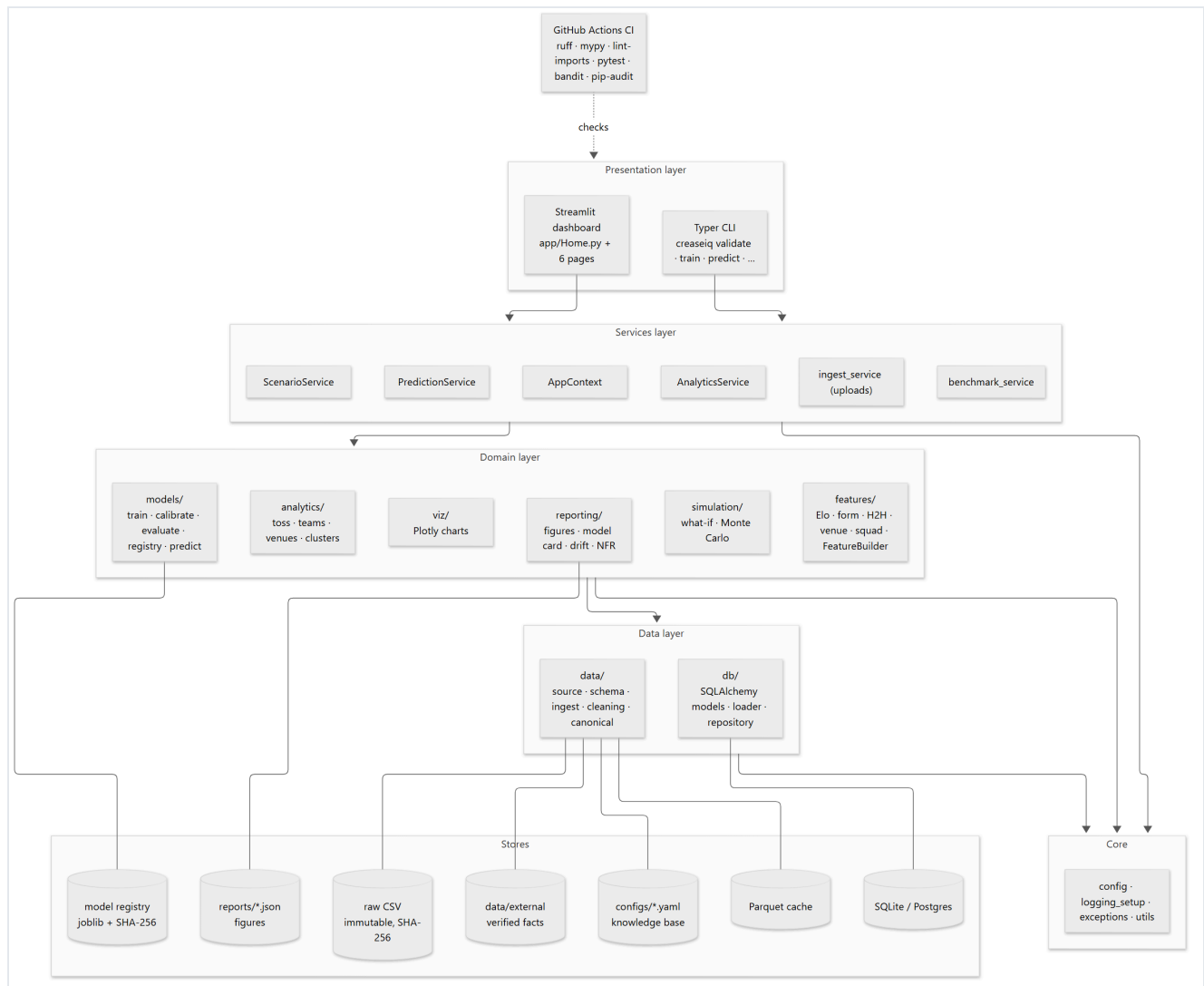


Figure 1: System architecture (D1): layers, stores and CI

Table 7: Layers and responsibilities

Layer	Packages	Responsibility
4 Presentation	app, cli	Widgets and commands, rendering, friendly errors
3 Services	services	Use cases: validate, orchestrate, log, time
2 Domain	analytics, features, models, simulation, reporting, viz	Statistics, feature engineering, machine learning, simulation, figures
1 Data access	data, db	Sources, schema, cleaning, canonical maps, ORM, repositories
0 Core	config, logging_setup, exceptions, utils	Cross-cutting concerns

How the layering is enforced. An `import-linter` contract runs in CI, and an AST test (`tests/unit/test_architecture.py`) fails if any module imports a higher layer. The benefits:

- The dashboard and the CLI share one implementation of every use case.
- Domain code is testable without the UI.
- A new front end (for example a REST API) would only add a layer-4 module.

Data stores:

- **Raw CSV.** Immutable; its SHA-256 is checked on every run.
- **Verified external facts.** Super-over winners and the D/L list, with their sources.
- **YAML knowledge base.** Franchise lineage, venues and home grounds.
- **Parquet cache.**
- **SQLite database.** Switchable to PostgreSQL by URL.
- **Hash-verified model registry.**
- **Generated reports and figures.**

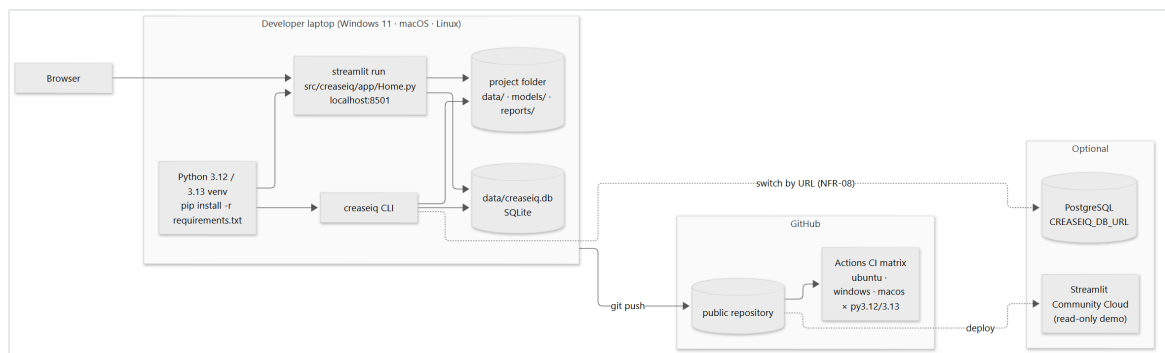


Figure 2: Deployment and environment (D10)

6.1 Technology stack

Table 8: Technology choices

Concern	Choice	Reason
Language	Python 3.12 / 3.13	Current numpy and scipy require ≥ 3.12 (ADR-002)
Data	pandas 3, NumPy, pyarrow	Standard; columnar cache
Validation	pandera	Declarative schemas with lazy failure collection
Statistics	SciPy, statsmodels	Exact tests; cross-checked results
ML	scikit-learn	All model families without native OpenMP dependencies
Storage	SQLAlchemy 2 + SQLite	Zero administration; Postgres available by URL
UI	Streamlit + Plotly	Python-only interactive dashboard, testable with <code>AppTest</code>
CLI	Typer + Rich	Typed commands with <code>--help</code>
Quality	pytest, Hypothesis, ruff, mypy, import-linter, bandit, pip-audit	Automated gates in CI
Docs	Mermaid, matplotlib, Playwright/Chromium	Diagrams as code; the same renderer on all operating systems

7. Design Diagrams

All diagrams are written as Mermaid sources in `docs/diagrams/` and drawn from the actual code: class names, table columns and import rules match the source files.

7.1 Use case diagram

Three actors use the system:

- **Analyst / student:** explore, analyse and export.
- **Cricket fan:** predict, what-if and simulate.
- **Maintainer:** ingest, append, rebuild, train and benchmark.

Each use case carries its FR identifiers. Mermaid has no native use-case notation, so actors are drawn as circles and use cases as rounded shapes inside the system boundary.

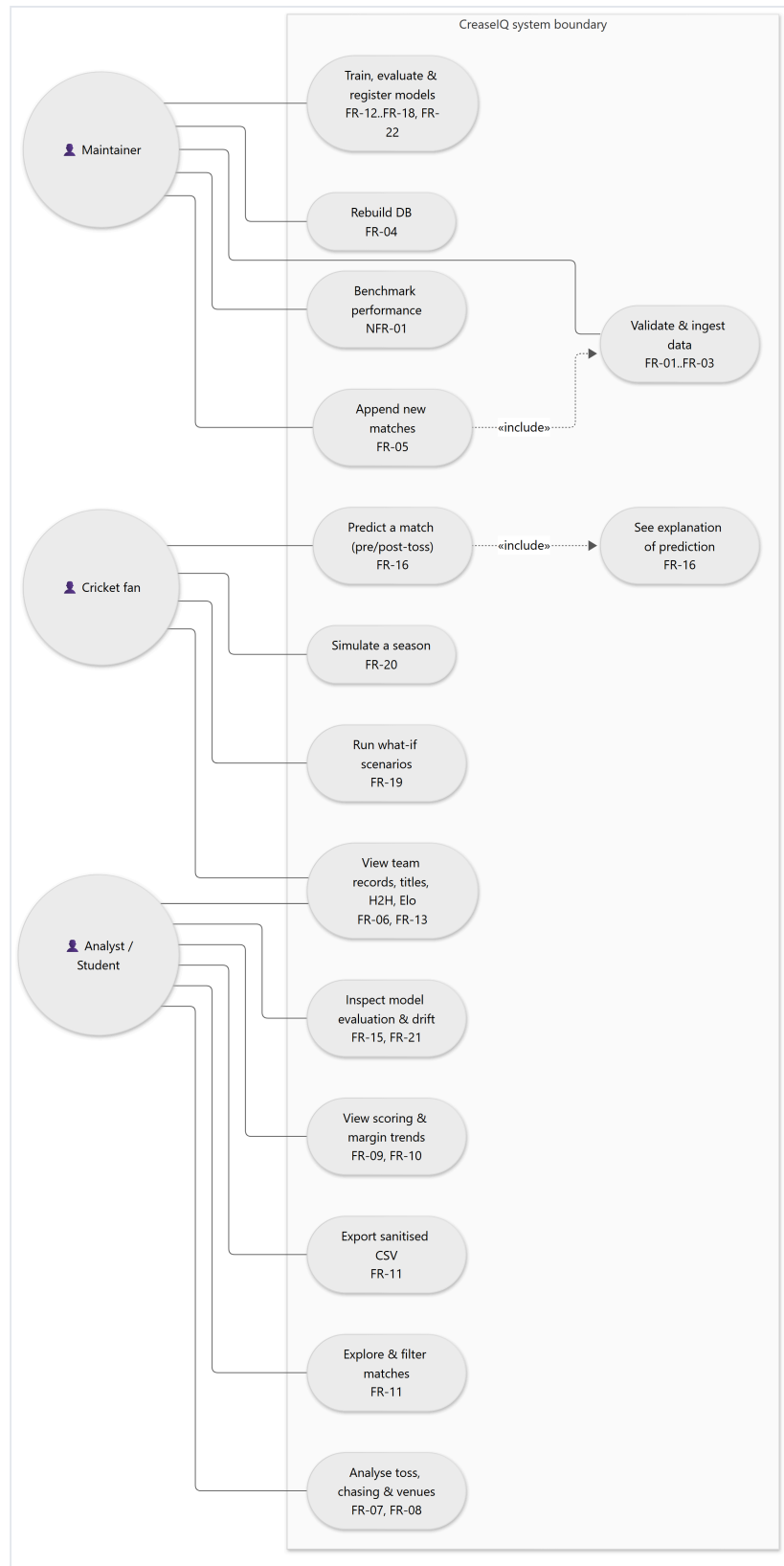


Figure 3: Use case diagram (D4)

7.2 Workflow diagrams

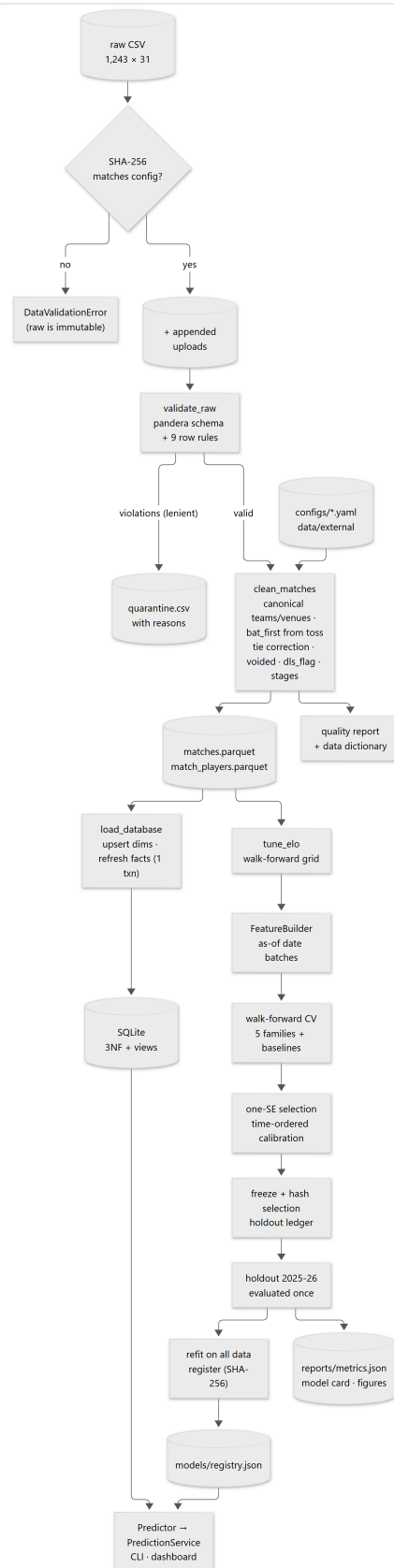


Figure 4: Process flow: the data and modelling pipeline run by `creaseiq all` (D2)

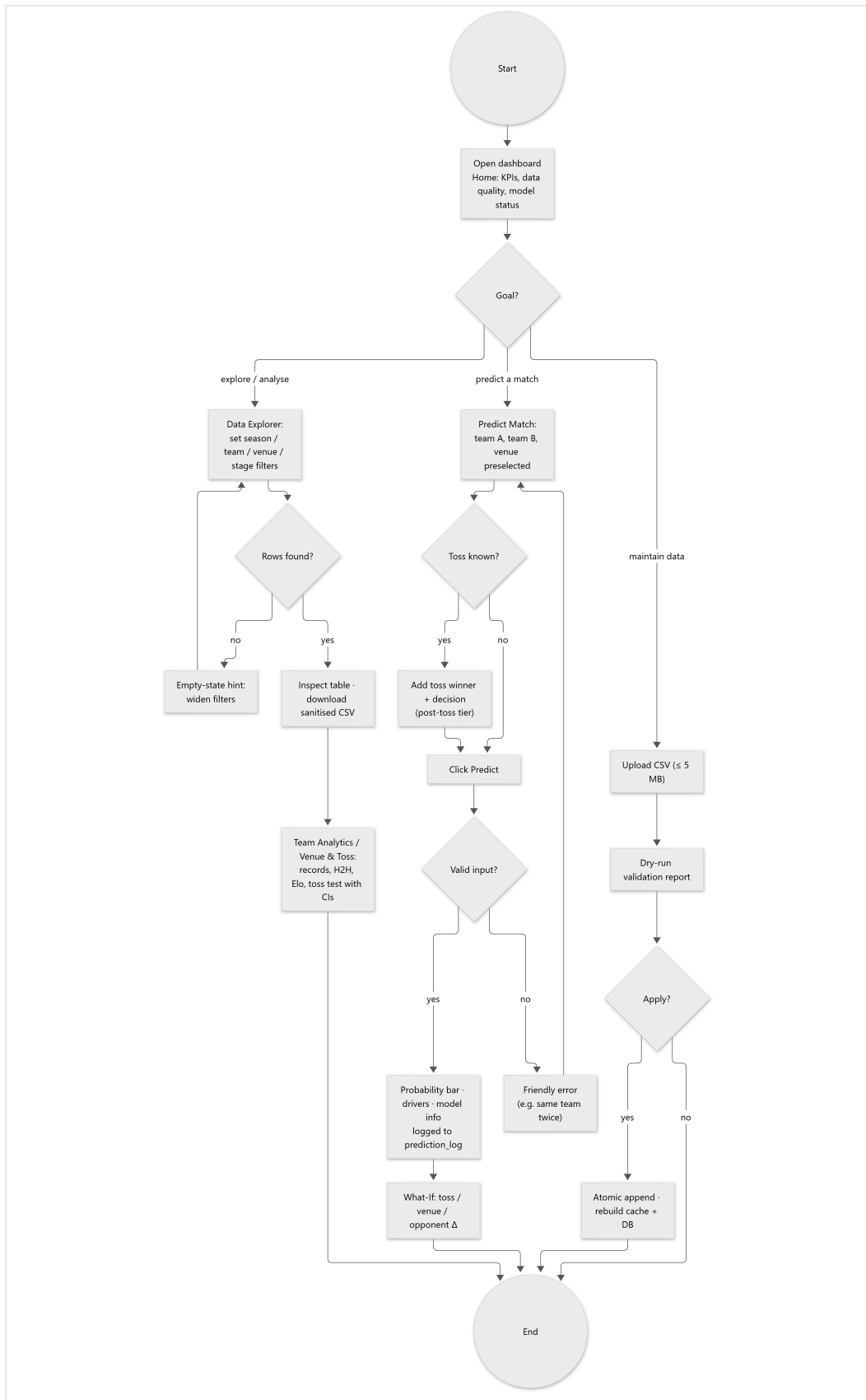


Figure 5: User workflow (activity diagram): analyst, predictor and maintainer paths (D3)

7.3 Sequence diagrams

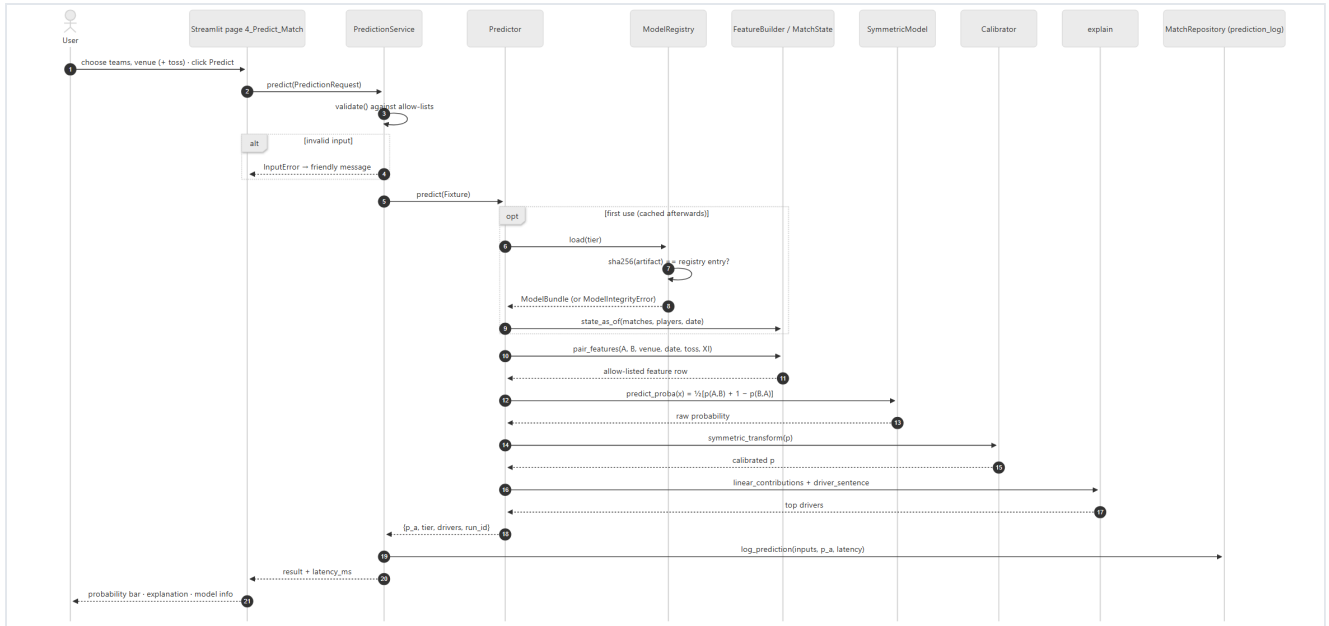


Figure 6: Sequence: serving one prediction, from validation to the prediction log (D5)

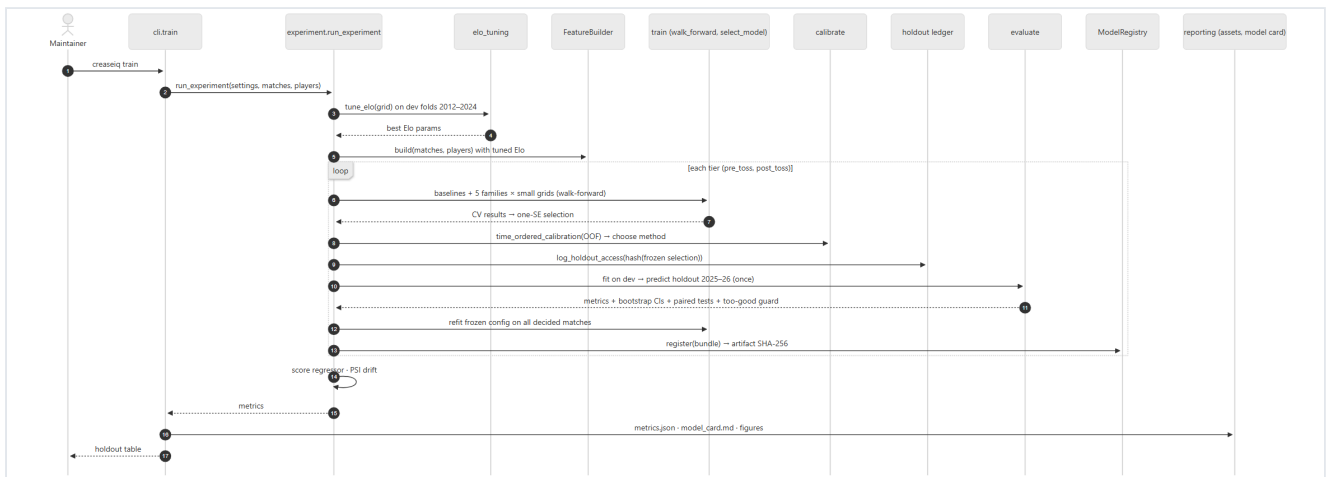


Figure 7: Sequence: training, selection, one-shot holdout evaluation and registration (D6)

7.4 Class and component diagrams

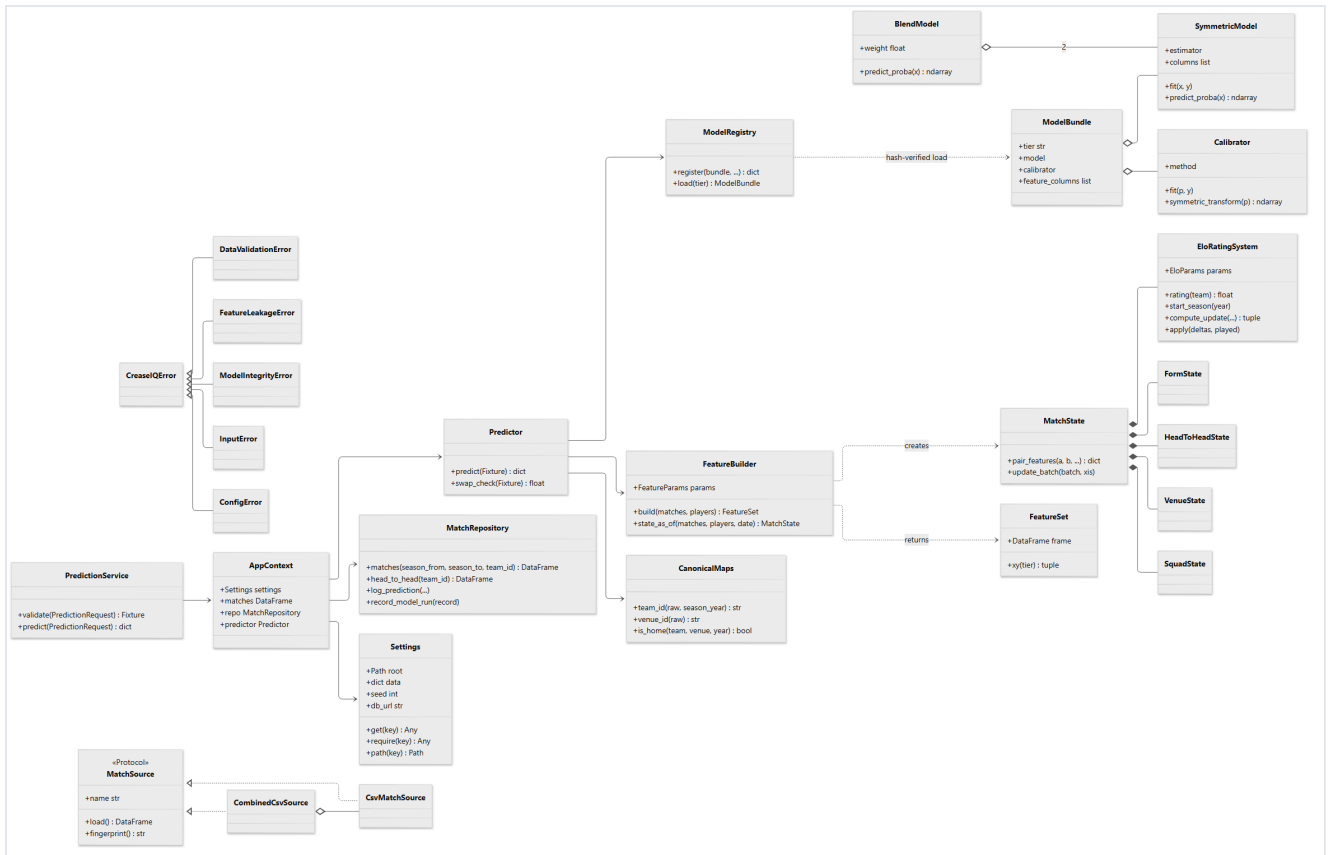


Figure 8: Class diagram of the key classes (D7)

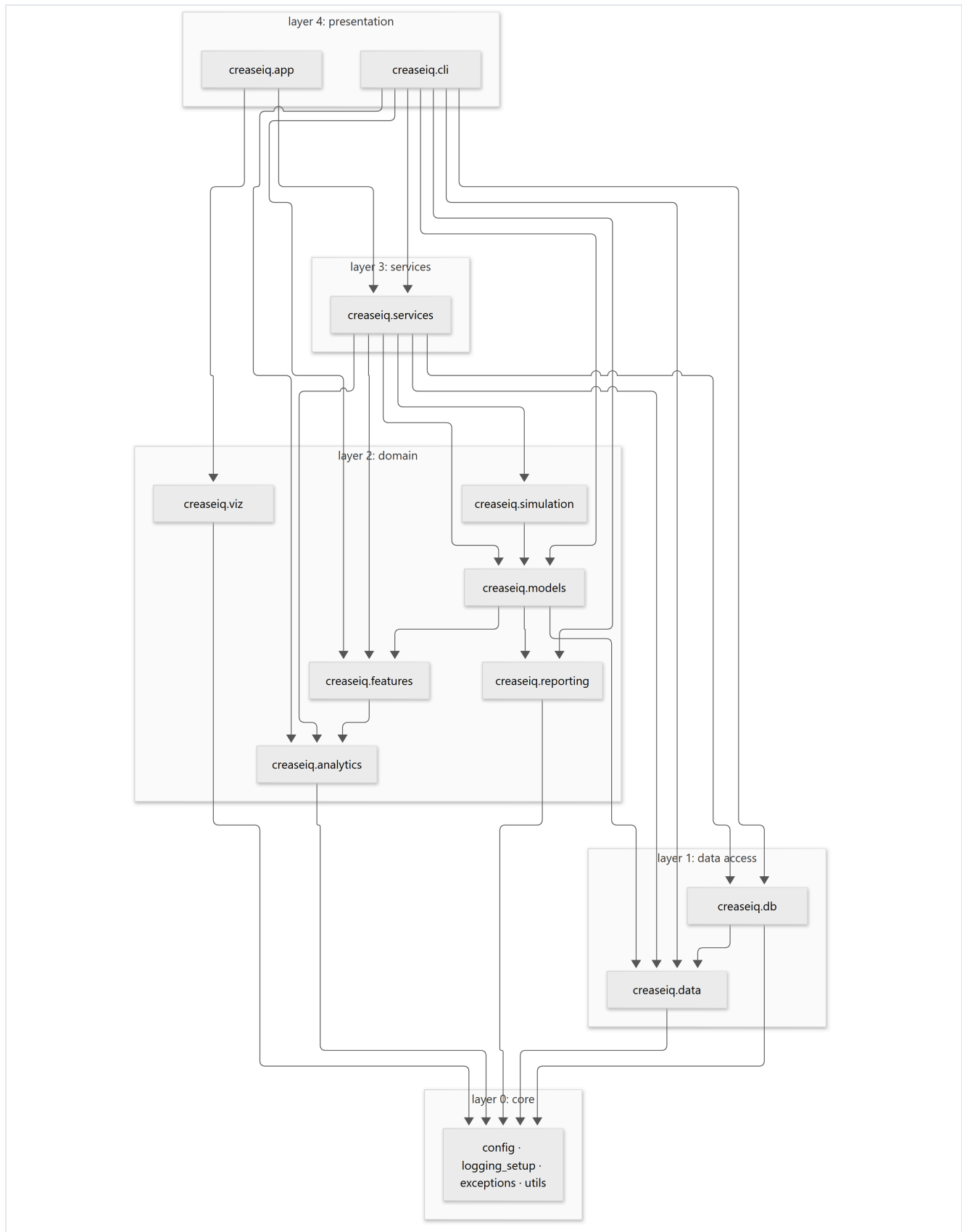


Figure 9: Component / package diagram; arrows are the only allowed imports (D8)

7.5 ER diagram and schema

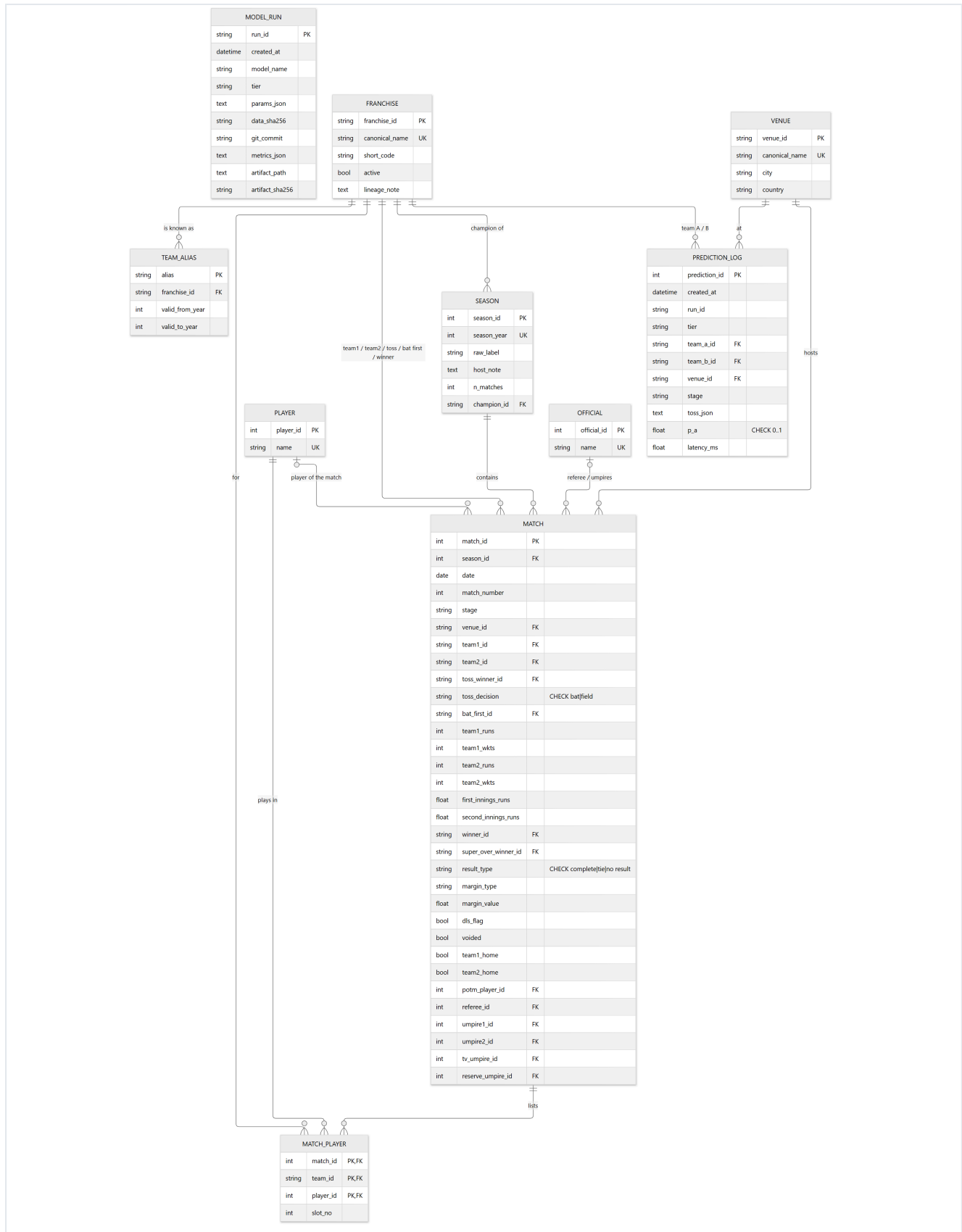


Figure 10: Entity-relationship diagram generated from the ORM models (D9)

Schema summary. The schema has ten tables and two views:

- **Dimension tables:** franchise, team_alias, venue, season, player, official.

- **Fact tables:** `match` and the associative `match_player`.
- **Operational tables:** `model_run` and `prediction_log`.
- **Views:** `v_team_season_summary` and `v_head_to_head`.

Constraints. The database enforces:

- a natural key, `UNIQUE(date, team1_id, team2_id)`;
- `CHECK` s on the toss decision, result type, distinct teams, "winner present iff complete" and probabilities in `[0, 1]`;
- foreign keys, with `PRAGMA foreign_keys=ON`.

The full DDL is generated from the models into `docs/schema.sql`.

Normalisation (3NF). Every non-key attribute depends on the key, the whole key and nothing but the key:

- Franchise names live only in `franchise`, and aliases (with validity years) in `team_alias`.
- Venue attributes live only in `venue`.
- Player and official names live in their own tables, referenced by surrogate ids.
- Squads form a many-to-many relation (`match_player`, composite key `match_id, team_id, player_id`), not a repeating group.
- `match` holds only foreign keys and attributes of the match itself.

The one deliberate redundancy is `season.champion_id`, which can be derived from the season's final. It is kept as a documented, test-verified convenience.

7.6 Model lifecycle

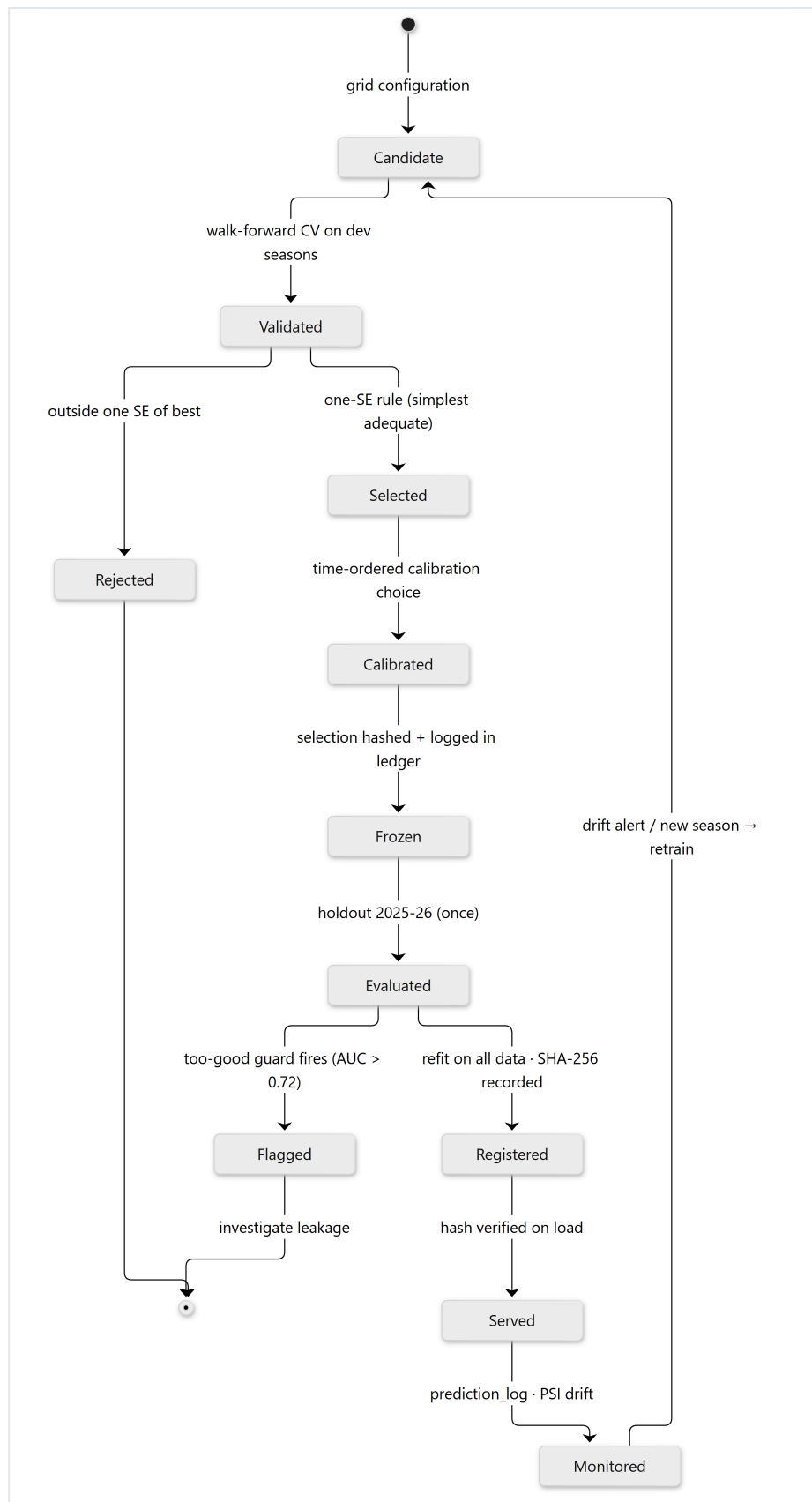


Figure 11: Model lifecycle state diagram (D12)

8. Design Decisions & Rationale

Non-obvious decisions are recorded as Architecture Decision Records (ADRs) in `docs/decisions/`. Each ADR gives the context, the options considered, the decision and its consequences. They are summarised below.

Table 9: Architecture decision records

ADR	Decision	Why
001 Course relevance	ML core, with AI fundamentals mapped explicitly; adds k-means venue profiling	An irrelevant project scores 0; the syllabus covers agents, search and knowledge representation as well as ML
002 Stack	Python ≥ 3.12 ; scikit-learn only; Playwright for PDFs	numpy 2.5 and scipy 1.18 dropped 3.11; avoids native OpenMP and GTK installs on Windows
003 Franchise continuity	Renames share one id; new owner or contract means a new franchise	Legally correct; fixes title counts and head-to-head; sensitivity documented
004 Data corrections	Correct ties with regulation scores, flag voided and D/L rows; never quarantine them	Keeps 16 real matches; the raw values stay available for audit
005 Orientation & symmetry	Hash-seeded coin flip per match; declared antisymmetric features; symmetrised predictions	Raw <code>team1</code> order leaks batting order from 2018
006 Selection, calibration, holdout	Walk-forward CV; one-SE rule; time-ordered calibration; hashed one-shot holdout with a ledger	Prevents overfitting noise and silent re-use of the holdout

8.1 The orientation trap (ADR-005)

A classifier needs a target such as "does team A win?", so every match must be oriented. The obvious choice, raw `team1` as A, is wrong. From 2018 onward `team1` is always the side batting first, and batting first wins only 45.3% of decided matches. A model trained on that orientation would learn batting order through a side channel, even in the pre-toss tier.

CreaselQ handles this in four steps:

1. Each match is oriented by a SHA-256-seeded coin flip on its own date and teams, so deleting other matches cannot change it.
2. Every feature is declared *antisymmetric* (a difference or ± 1 flag) or *symmetric context* (venue, stage, era).
3. The logistic model has no intercept and uses only antisymmetric features. That makes it exactly symmetric: $p(A,B) = 1 - p(B,A)$.
4. Tree models are symmetrised at prediction time, and a test checks invariance to $1e-9$.

8.2 Leakage-safe features ("as-of")

Features are built by one incremental state machine that processes matches in **date batches**. Every match on a date is featurised from the state *before* that date, and the state is updated only after the whole date. Same-day double-headers therefore never see each other. Five automated tests guard this (Section 11): future deletion, future mutation, same-day isolation, label shuffle, and the column allow-list.

8.3 Why walk-forward, the one-SE rule and log-loss

Random k-fold would train on future seasons. The IPL is non-stationary (auctions, the Impact Player rule, neutral seasons), and Cerqueira et al. recommend time-ordered out-of-sample evaluation for such series

[11]. CreaseIQ therefore validates each development season 2012–2024 with a model trained on all earlier seasons.

With only about 1,000 development matches, the gaps between models are small. The *one-standard-error rule* picks the simplest model within one SE of the best mean. The primary metric is **log-loss**, a proper scoring rule. Accuracy ignores confidence, and selecting on calibration is known to be more useful than selecting on accuracy [9].

8.4 Calibration choice

The candidates are none, Platt (sigmoid) and isotonic. Each is evaluated in time order: the calibrator for season *S* is fitted only on out-of-fold predictions from earlier seasons. Isotonic regression overfits below about 1,000 calibration points [12], [13], and the result confirmed it here:

Tier	none	sigmoid	isotonic	Chosen
post_toss	0.6852	0.6861	0.7209	none
pre_toss	0.6914	0.6918	0.7068	none

8.5 One-shot holdout with a ledger

2025–26 is the holdout. Before it is touched, the selection is frozen: model, parameters, calibration, Elo settings, feature list and data hash are hashed into a selection id. The id is logged in `reports/holdout_ledger.json`, which counts every evaluation.

Re-running the same frozen selection recomputes identical numbers. Evaluating a *different* selection would be flagged as holdout re-use. ADR-006 also discloses that the ledger file was reset once after an exploratory run of the same selection.

8.6 Other decisions

- **SQLite vs a server database:** zero administration for a single-user app; PostgreSQL remains one URL away (NFR-08).
- **Streamlit:** a Python-only UI, with `AppTest` for automated UI tests.
- **Correct, don't quarantine:** anomalies with a verifiable explanation are corrected. Unknown values fail loudly instead of being guessed.
- **Causal vs associational toss analysis:** the toss is randomised, so "toss winner wins" is a valid causal test. The bat/field *decision* is post-treatment, so tests conditioned on it are labelled associational [2].
- **Shrinkage:** venue and team-at-venue rates are pulled toward league values, so venues with few matches are not over-interpreted.

9. Implementation Details

9.1 Dataset description

The input is a Cricsheet-derived IPL match file [1]:

- **Size:** 1,243 rows × 31 columns, one row per match, from 2008-04-18 to 2026-05-31.
- **Integrity:** SHA-256 `db81a92c740ef6d9...`, stored immutably.
- **Columns:** event, season label, match number, date, city, venue, the two teams, toss winner and decision, runs and wickets for each team, winner, result type, margins, player of the match, five officials, six constant columns (T20, 20 overs, 6 balls, male, club), and the two squad lists.
- **Squads:** 11 players each through 2022, and 12 from 2023, when the Impact Player substitute is listed.

Table 10: Outcome composition

Matches	Decided	Ties	No result	Voided	Distinct players	Toss decision field / bat
1,243	1,218	16	9	1	811	825 / 418

Table 11: Seasons and derived champions (champions verified against published results for all seasons)

Season	Label	Matches	Decided	Ties	No result	Champion
2008	2007/08	58	58	0	0	rr
2009	2009	57	56	1	0	deccan_chargers
2010	2009/10	60	59	1	0	csk
2011	2011	73	72	0	1	csk
2012	2012	74	74	0	0	kkr
2013	2013	76	74	2	0	mi
2014	2014	60	59	1	0	kkr
2015	2015	59	56	1	2	mi
2016	2016	60	60	0	0	srh
2017	2017	59	58	1	0	mi
2018	2018	60	60	0	0	csk
2019	2019	60	57	2	1	mi
2020	2020/21	60	56	4	0	mi
2021	2021	60	59	1	0	csk
2022	2022	74	74	0	0	gt
2023	2023	74	73	0	1	csk
2024	2024	71	71	0	0	kkr
2025	2025	74	70	1	3	rcb
2026	2026	74	72	1	1	rcb

9.2 Validation and cleaning

Validation has two layers:

1. A pandera schema checks types, value ranges, allowed values and the six constant columns.
2. Nine vectorised row rules check cross-column logic:

- the teams are distinct;
- the toss winner and the winner both took part in the match;
- a winner is present exactly when the result is "complete";
- exactly one margin is recorded;
- there are no margins on ties;
- at most 10 wickets except on ties;
- the date is valid;
- each squad lists 11–13 players.

In lenient mode, failing rows go to `quarantine.csv` with named reasons. Strict mode raises with the row indices. The real file passes strict validation with **0 quarantined rows**.

Cleaning records every change it makes:

Table 12: Fixes applied by the cleaning pipeline (from the data-quality report)

Fix	Rows
city_corrected	170
city_unknown_imputed	51
dls_flag_external	16
dls_flag_heuristic	14
dls_flag_total	19
no_result_scores_excluded	9
player_lists_parsed	2,486
season_label_mapped	178
stage_derived_playoff	74
team_alias_canonicalised	651
tie_innings_corrected_super_over_removed	16
venue_alias_canonicalised	862
voided_flagged	1

Derived columns (all documented in `docs/data_dictionary.md`):

- `season_year` and batting order from the toss.
- Regulation first- and second-innings runs.
- An era-specific playoff `stage`, `season_champion` and `impact_era`.
- Home flags from a researched home-ground knowledge base, with neutral seasons covering 2009, 2020, 2021 and 2022.
- `dls_flag`, `voided` and `scores_usable`.

9.3 Feature engineering

Every feature is computed *as of the day before* the match (Section 8.2) and oriented A – B. The groups are:

Table 13: Feature groups (tier B adds the toss and squad groups)

Group	Features	Notes
Elo	<code>elo_diff</code> , <code>home_diff</code>	Margin-aware Elo, tuned; home bonus only when exactly one side is at home

Group	Features	Notes
Form	last-5 and last-10 smoothed win rate, runs scored and conceded (last 10), season win % and games, rest days	Smoothed toward 0.5 (prior strength 2)
Head-to-head	all-time and last-5-seasons edge, meetings	Edge = (wins_A – wins_B)/(n + 2)
Venue	team-at-venue edge, shrunk venue scoring level, shrunk bat-first rate	Priors are the league values <i>so far</i>
Context	playoff flag, Impact Player era	Symmetric
Toss (post-toss)	A bats first, A won the toss, batting order × venue, batting order × era	Interactions let linear models use venue and era context
Squad (post-toss)	experience of the XI, prior POTM awards, debutants, continuity (Jaccard)	Named XI known at the toss

Elo engine.

- **Expected score:** $1/(1 + 10^{-(R_A - R_B + H)/400})$.
- **Update:** $K \times \text{margin multiplier} \times (S - E)$, where the multiplier is $\ln(\text{units} + 1) \cdot 2.2 / (0.001 \cdot \Delta_{\text{winner}} + 2.2)$ [6].
- **Margin units:** runs/10 or wickets/2.
- **D/L results:** the multiplier is 1.
- **Season carry-over:** ratings regress toward 1500 each season.

The Elo parameters were tuned on development folds over 400 combinations. Best: `home_bonus=75`, `k=5`, `season_regression=0.1`, `use_margin=True`, with walk-forward log-loss 0.6900 against 0.6910 for the defaults.

9.4 Model selection rationale

With about 1,200 matches and weak signal, the model families are deliberately simple and their search grids small:

Table 14: Walk-forward mean log-loss of the best configuration of each family, with baselines (development seasons)

Candidate	Pre-toss	Post-toss
elo_logit	0.6900 ± 0.0172	0.6900 ± 0.0172
logreg	0.6908 ± 0.0085	0.6857 ± 0.0126
random_forest	0.6924 ± 0.0117	0.6906 ± 0.0127
hist_gb	0.6949 ± 0.0138	0.6909 ± 0.0195
blend	0.6909 ± 0.0097	0.6862 ± 0.0138
B0 coin flip	0.6931	0.6931
B3 Elo probability	0.6920	0.6920
B1 chase prior	n/a	0.6902
B2 toss winner	n/a	0.6940
B4 venue chase rate	n/a	0.6993

The families:

- **Elo-logit.** Interpretable, with two features.
- **L2 logistic regression.** No intercept and antisymmetric features only (Section 8.1).
- **Random forest and histogram gradient boosting.** They can use the symmetric context through interactions.

- **Blend.** Of logistic regression and gradient boosting.

Selection used the one-SE rule:

- **Pre-toss: elo_logit** $C=1.0$. The best mean belonged to elo_logit.
- **Post-toss: logreg** $C=0.003$. The best mean belonged to logreg.

The tree ensembles did not beat the linear models. That is expected with $n \approx 1k$ and weak signal, where variance dominates.

9.5 Evaluation methodology

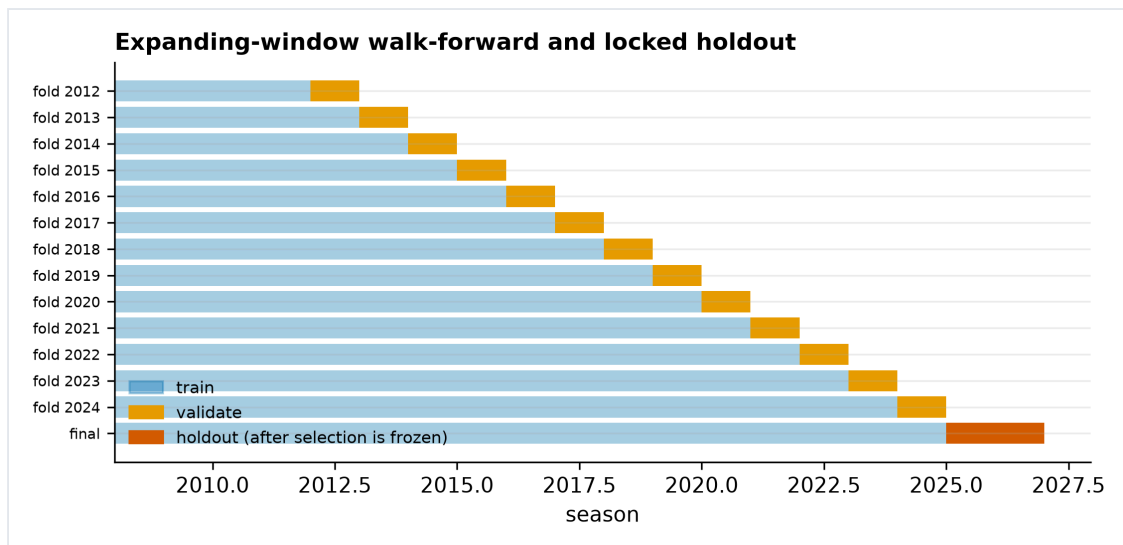


Figure 12: Expanding-window walk-forward folds and the locked holdout (D11)

- **Splits:**
 - Development seasons are ≤ 2024 , with folds validating seasons 2012–2024.
 - The holdout is 2025, 2026: $n = 142$ decided matches, evaluated once per frozen selection.
- **Metrics:**
 - Primary: log-loss.
 - Also: Brier score with its Murphy decomposition, accuracy, ROC-AUC, and ECE on equal-mass bins (descriptive only [14]–[16]).
- **Uncertainty:** a percentile bootstrap (2,000 resamples, seed 42) for every holdout metric.
- **Comparison with baselines:** a paired bootstrap of per-match log-loss differences (10,000 resamples), plus the Diebold–Mariano test with the Harvey–Leybourne–Newbold correction [18].
- **Baselines:** all rates are fitted on the training fold only. The plan's all-data chase rate would have leaked the holdout base rate (plan review finding 6).
- **Guards:**
 - A "too good" check flags holdout AUC above 0.72 or accuracy above 68% as suspected leakage.
 - Ablations add feature groups cumulatively.
 - Performance is broken down by era and stage.

9.6 Module walkthrough

Table 15: Packages and their responsibilities

Package	Main modules	Responsibility
data	source, schema, ingest, canonical, cleaning, quality_report, pipeline	FR-01..03: validated, canonical match table
db	models, session, loader, repository	FR-04: 3NF schema, idempotent load, parameterised queries
analytics	hypothesis_tests, toss_analysis, team_stats, venue_stats, venue_clusters, season_trends, player_stats, summary	FR-06..10
features	elo, form, head_to_head, venue_effects, squad, builder	FR-12, FR-13
models	splits, baselines, train, calibrate, evaluate, elo_tuning, explain, registry, predict, score_regressor, experiment	FR-14..18
simulation	what_if, season_monte_carlo	FR-19, FR-20
services	context, prediction_service, analytics_service, scenario_service, ingest_service, benchmark_service	Use cases for the CLI and UI
reporting	figures, model_card, drift, nfr, readme, report_builder, assets	FR-21, FR-22, this report
viz, app, cli	Plotly charts; 7 Streamlit pages; 15 CLI commands	Presentation

Besides `version` and `info`, the CLI commands are `validate`, `build-db`, `analyze`, `features`, `train`, `evaluate`, `predict`, `whatif`, `simulate`, `ingest`, `benchmark`, `report` and `all`. Expected errors exit with code 1 and a one-line message.

10. Screenshots / Results

10.1 Application screenshots

All screenshots are real captures of the running Streamlit app, taken with Playwright (`scripts/capture_screenshots.py`).

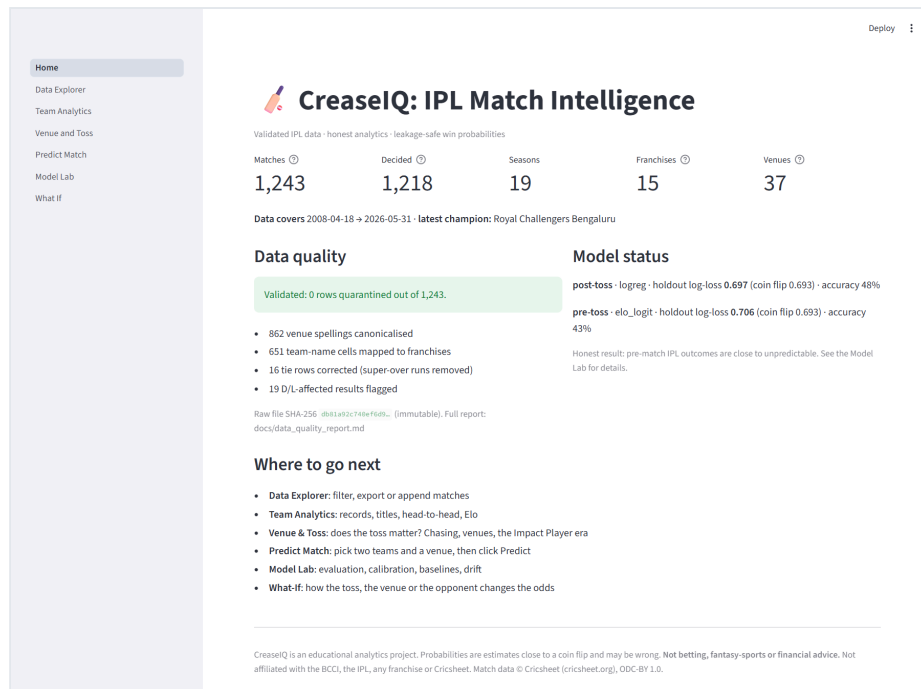


Figure 13: Home: KPIs, data-quality and model status

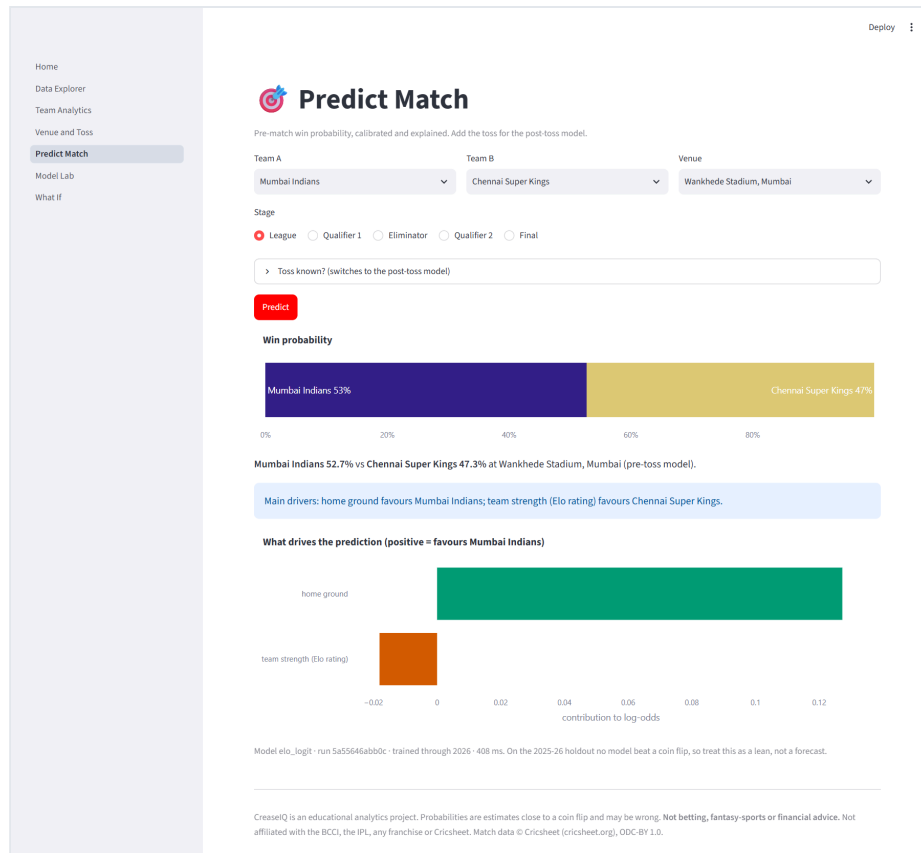


Figure 14: Predict Match: probability, plain-English drivers and model information

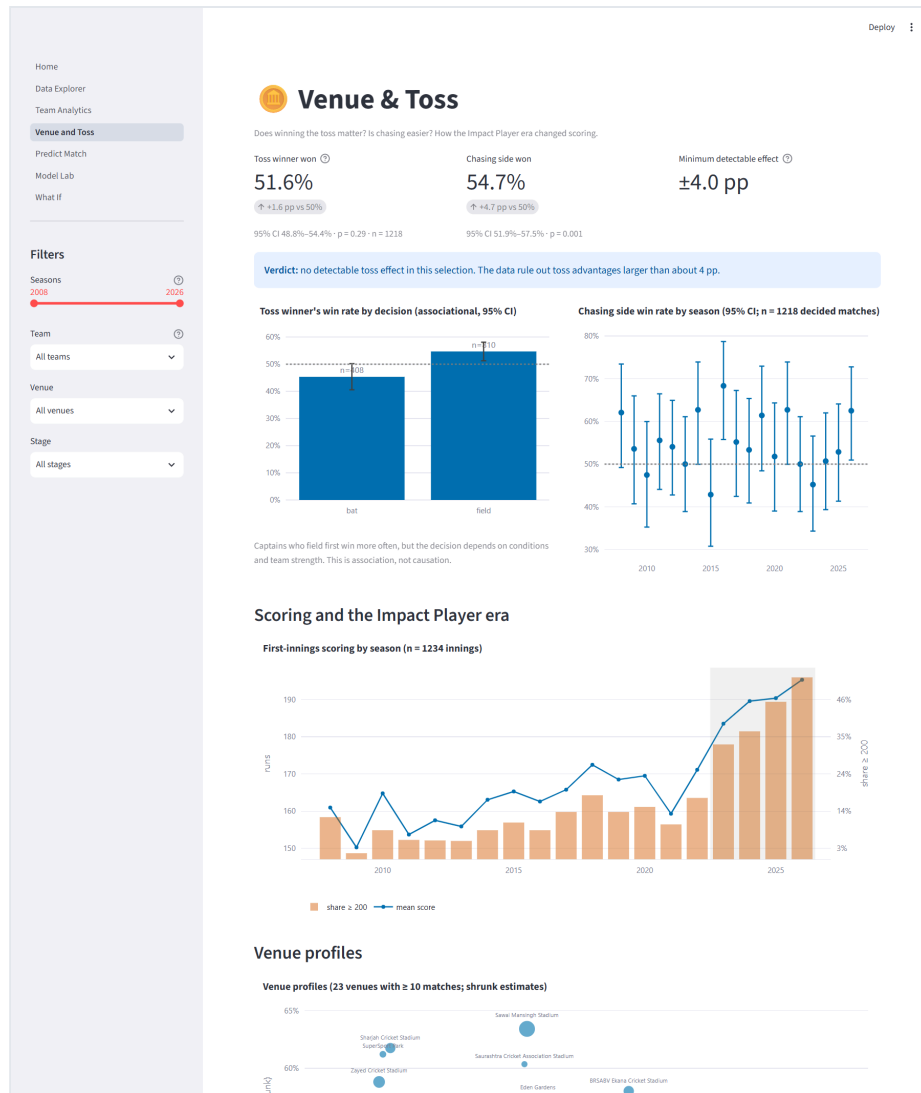
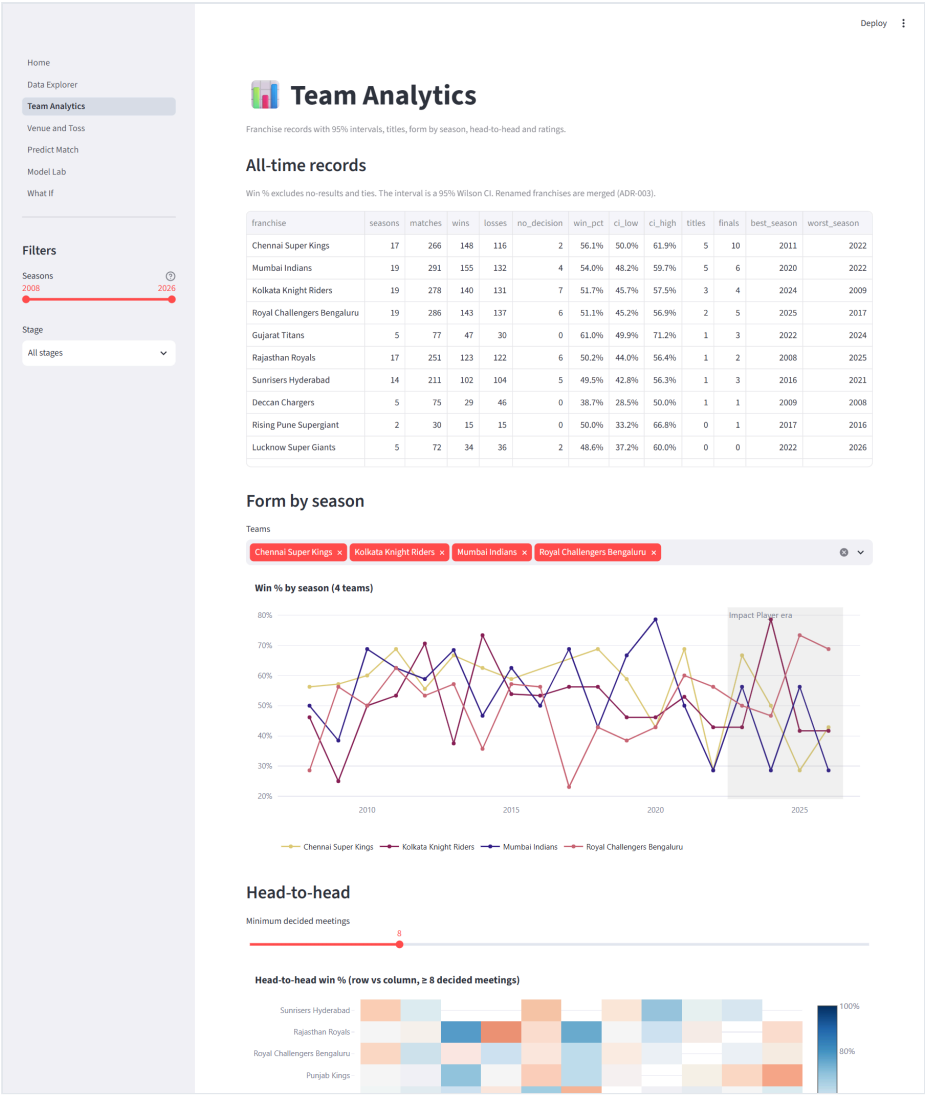
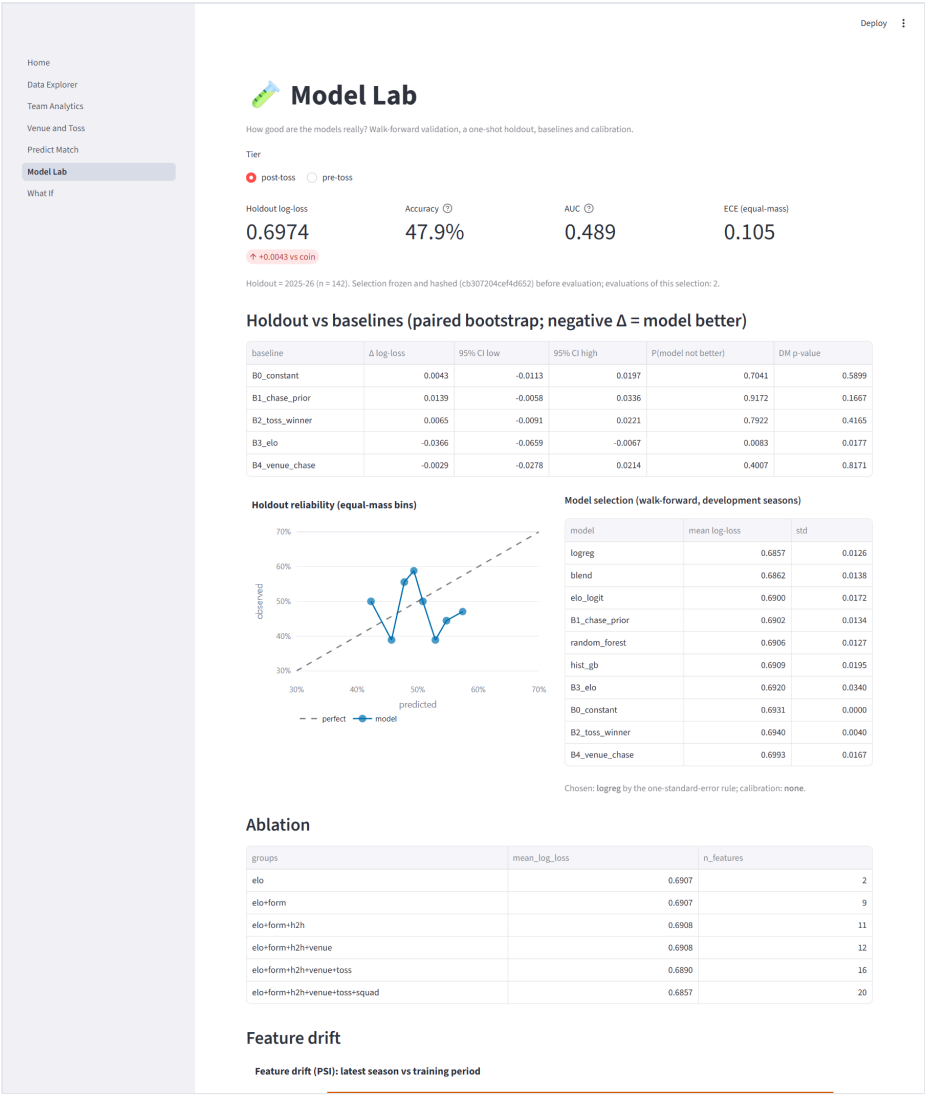


Figure 15: Venue & Toss: causal toss test with CI and minimum detectable effect, chasing by season





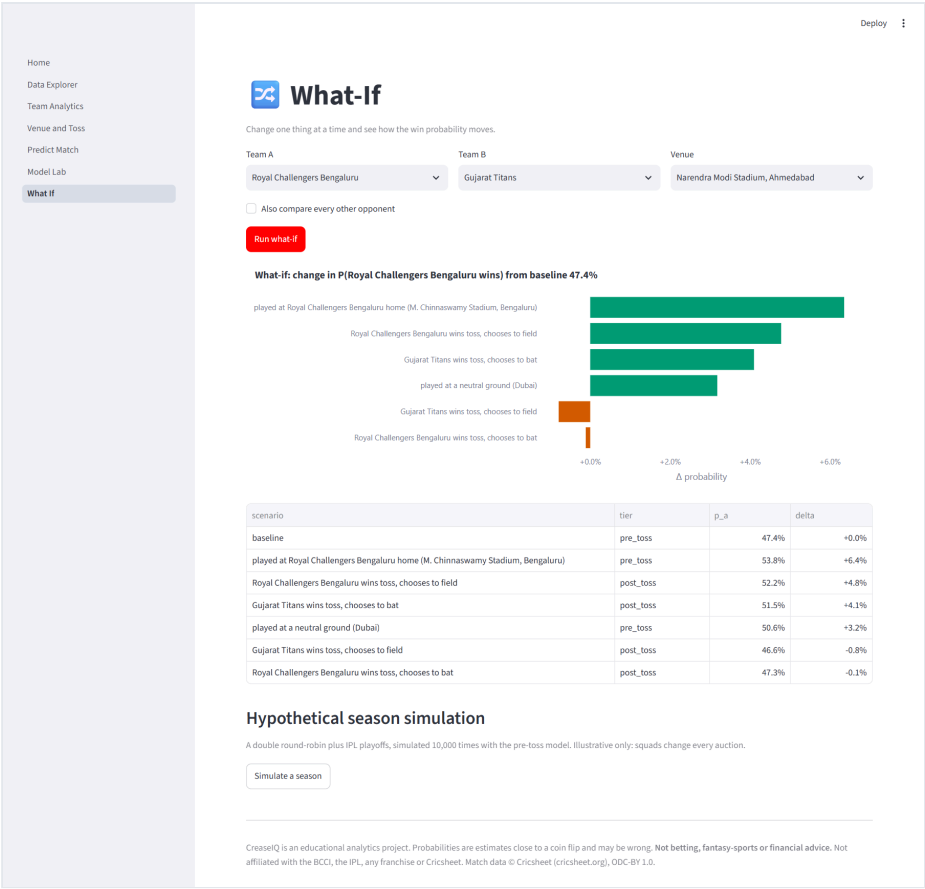


Figure 18: What-If: change in win probability for toss and venue scenarios

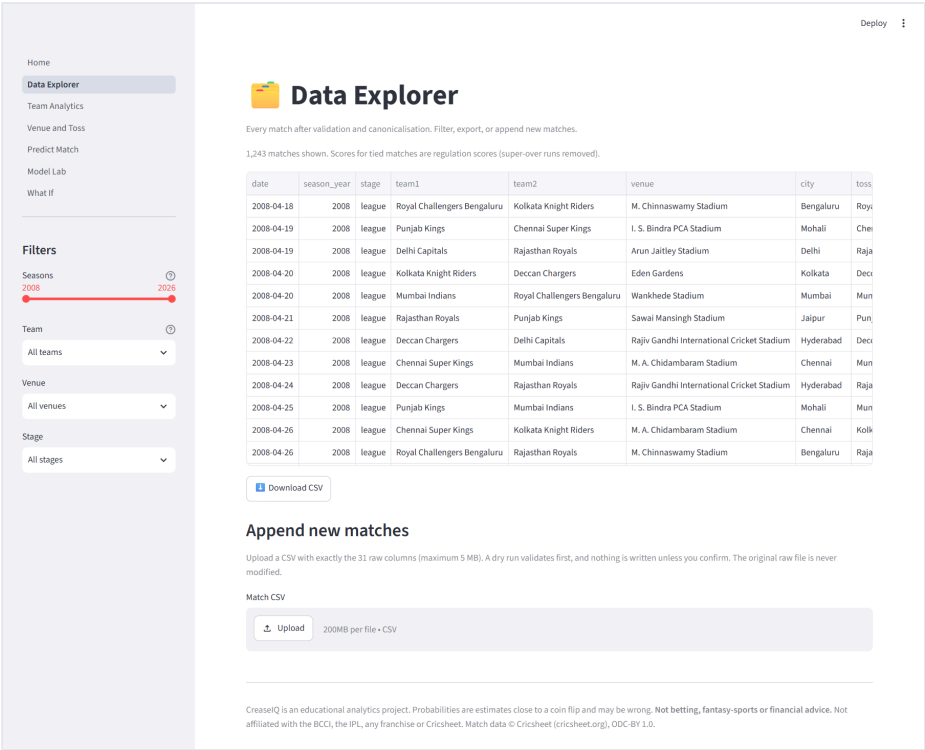


Figure 19: Data Explorer: filters, sanitised export and validated upload

10.2 Analytics highlights

Table 16: Headline statistical findings (decided matches)

Question	Estimate	95% CI	Test	Interpretation
Does the toss winner win?	51.6%	48.8%–54.4%	exact binomial $p = 0.29$	No detectable effect; MDE ± 4.0 pp
Does the chasing side win?	54.7%	51.9%–57.5%	$p = 0.001$	Real, modest chasing edge
Chase edge, Impact era minus pre-2023	-2.5 pp	-9.1 to 4.1 pp	z-test $p = 0.46$	No evidence it changed
Home side wins (neutral seasons excluded)	53.2%	50.0%–56.5%	$p = 0.057$	Small, borderline
First-innings runs, Impact era vs before	189.7 vs 162.6	$\Delta 22.6$ –31.7	Welch $p = 2.4e-27$; $d = 0.82$	Large effect
200+ first innings	41.7% vs 10.1%	n/a	n/a	4.1× as frequent

The toss winners who chose to field won more often (54.7% vs 45.3%; χ^2 $p = 0.002$; Cramér's $V = 0.09$). This is **associational**: captains choose to field when conditions and team strength favour it.

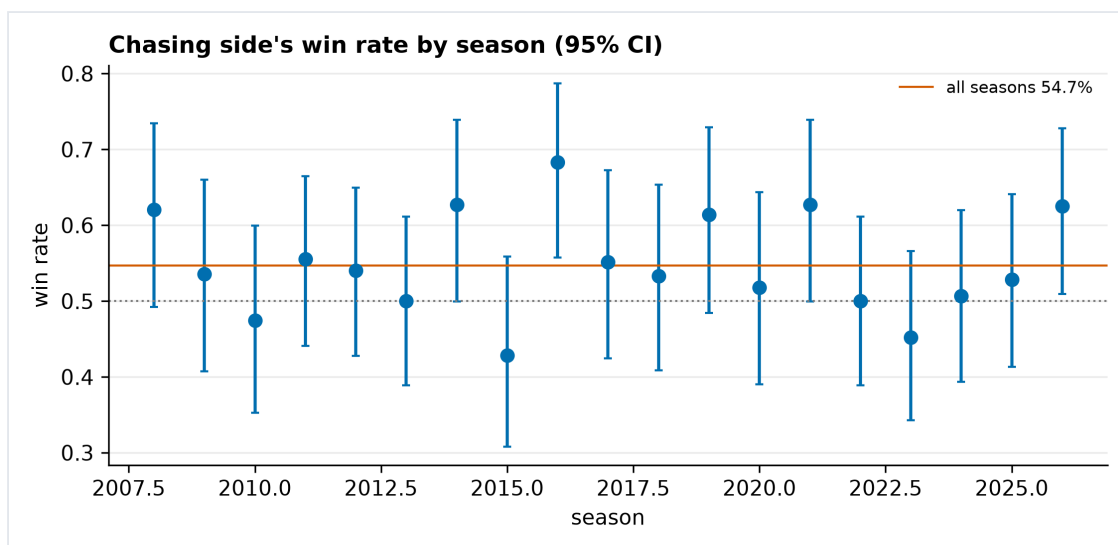


Figure 20: Chasing side's win rate by season with 95% Wilson intervals

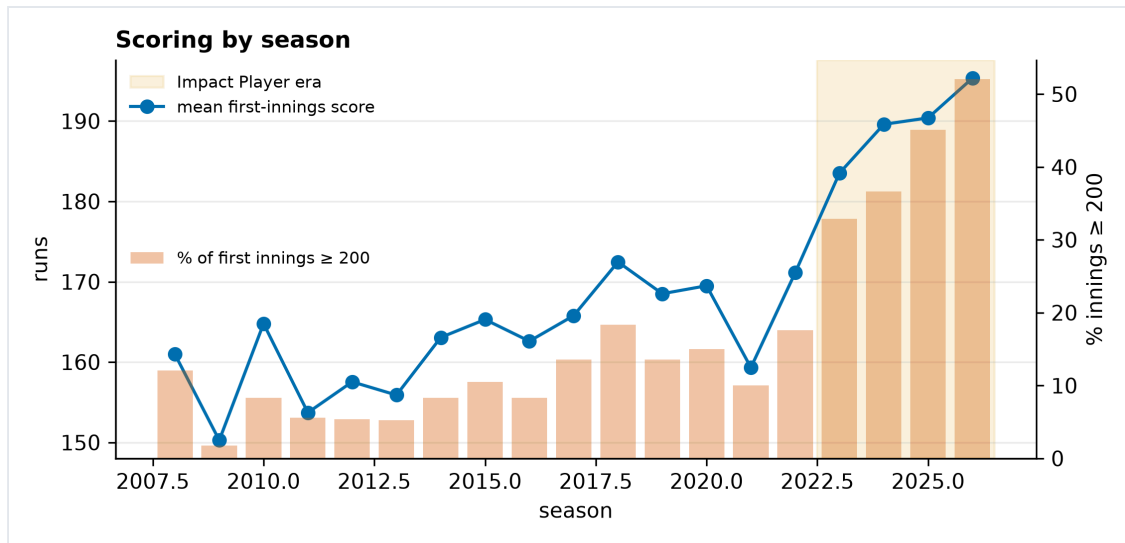


Figure 21: Mean first-innings score and share of 200+ totals; the Impact Player era is shaded

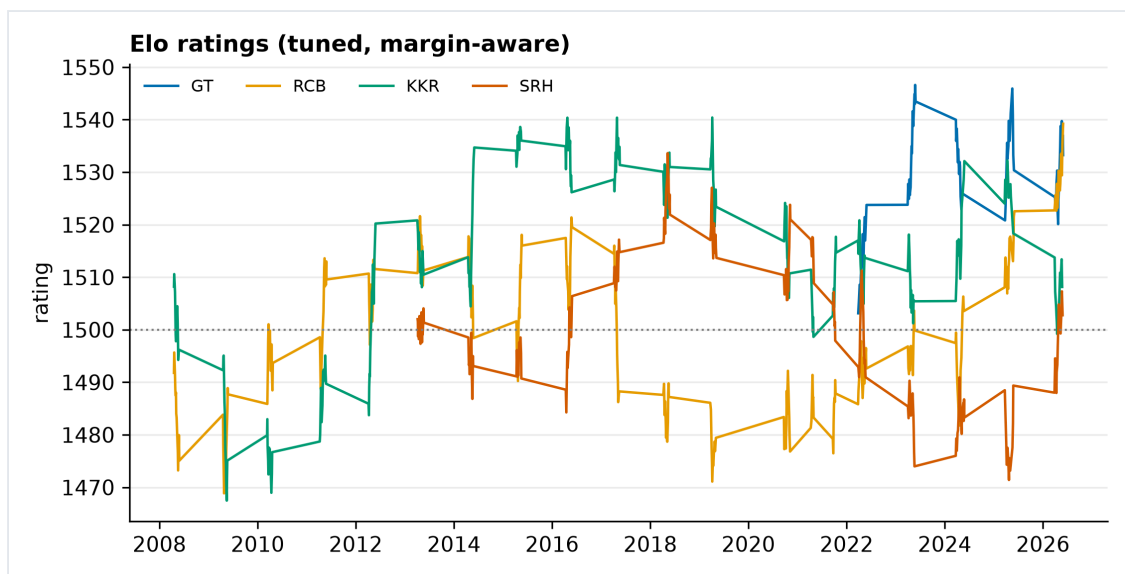


Figure 22: Tuned Elo ratings of the current top franchises

The unsupervised venue profiling chose $k = 2$ clusters by silhouette score: "higher-scoring, less chase-friendly", "lower-scoring, more chase-friendly".

10.3 Win-probability results

Table 17: Holdout (2025–26) metrics with 95% bootstrap CIs

Tier	Model	Calibration	n	Log-loss	Brier	Accuracy	AUC	ECE
post_toss	logreg	none	142	0.6974 [0.682, 0.714]	0.2522	47.9% [40.1%, 56.3%]	0.489 [0.39, 0.59]	0.105
pre_toss	elo_logit	none	142	0.7060 [0.689, 0.723]	0.2564	43.0% [34.5%, 51.4%]	0.453 [0.35, 0.55]	0.120

Table 18: Holdout comparison with every baseline ($\Delta = \text{model} - \text{baseline log-loss}$; negative means the model is better)

Tier	Baseline	Baseline log-loss	Δ	95% CI of Δ	P(model not better)	DM p
post_toss	B0_constant	0.6931	+0.0043	[-0.0113, +0.0197]	0.70	0.59
post_toss	B1_chase_prior	0.6835	+0.0139	[-0.0058, +0.0336]	0.92	0.17
post_toss	B2_toss_winner	0.6910	+0.0065	[-0.0091, +0.0221]	0.79	0.42

Tier	Baseline	Baseline log-loss	Δ	95% CI of Δ	P(model not better)	DM p
post_toss	B3_elo	0.7340	-0.0366	[-0.0659, -0.0067]	0.01	0.02
post_toss	B4_venue_chase	0.7003	-0.0029	[-0.0278, +0.0214]	0.40	0.82
pre_toss	B0_constant	0.6931	+0.0129	[-0.0043, +0.0299]	0.93	0.14
pre_toss	B3_elo	0.7340	-0.0280	[-0.0487, -0.0068]	0.01	0.01

Interpretation. In walk-forward validation on development seasons, the chosen post-toss model (0.6857) beats the coin flip (0.6931) and the chase prior (0.6902). On the 2025–26 holdout, however, **neither tier beats the constant 0.5 forecast**, and every confidence interval is wide at $n = 142$. The model was not re-tuned after this result. Home-side win rates since 2023 have been 2023: 40%, 2024: 58%, 2025: 40%, 2026: 52%, against 53% across all seasons, so relationships learned before the Impact Player era weakened. The "too good" leakage guard was not triggered. This is the realistic ceiling that a leakage-free evaluation reveals.

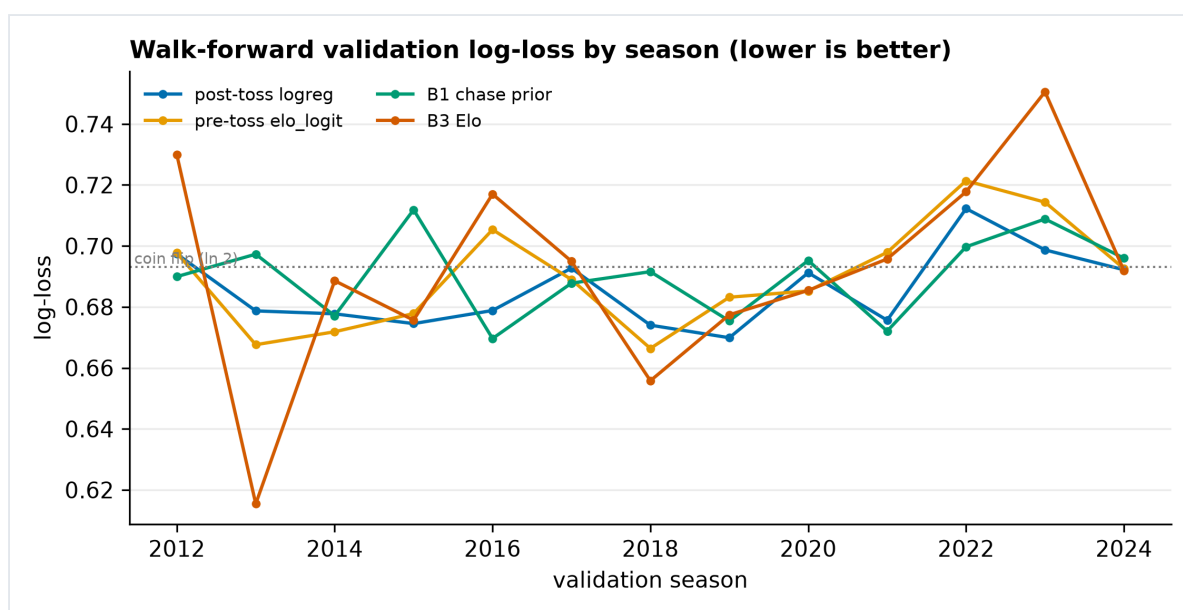


Figure 23: Validation log-loss by season: chosen models against baselines

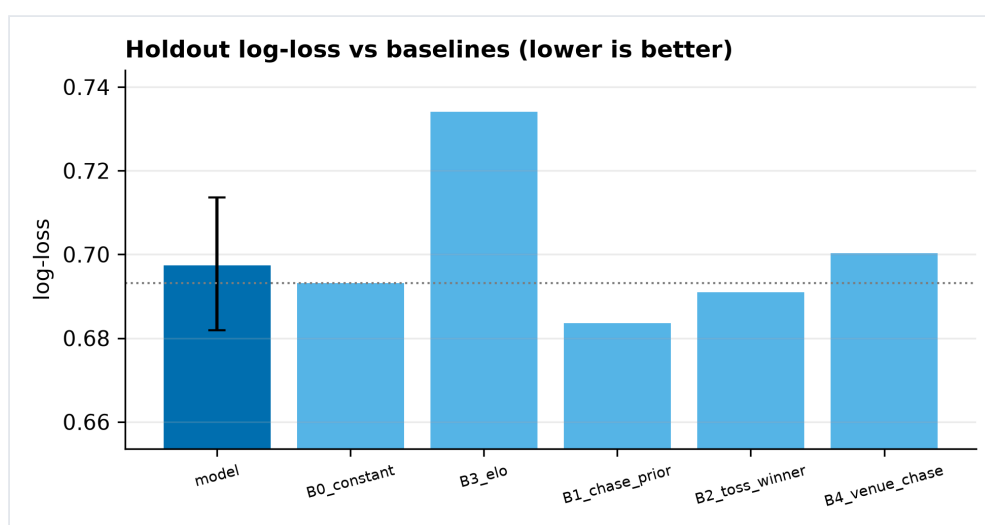


Figure 24: Post-toss holdout log-loss vs baselines (model bar with 95% CI)

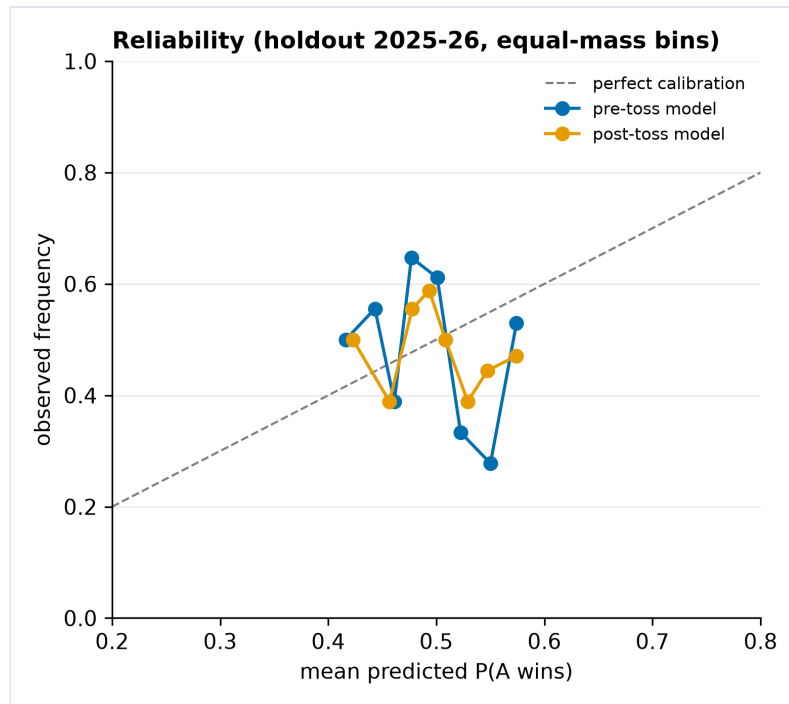


Figure 25: Holdout reliability diagram (equal-mass bins)

Table 19: Cumulative feature-group ablation (post-toss logistic, walk-forward mean log-loss)

Feature groups	# features	Log-loss
elo	2	0.6907
elo+form	9	0.6907
elo+form+h2h	11	0.6908
elo+form+h2h+venue	12	0.6908
elo+form+h2h+venue+toss	16	0.6890
elo+form+h2h+venue+toss+squad	20	0.6857

The ablation shows that most of the (small) gain comes from the **toss/batting-order** and **squad** groups, which are only known at the toss. History-only features barely move the needle.

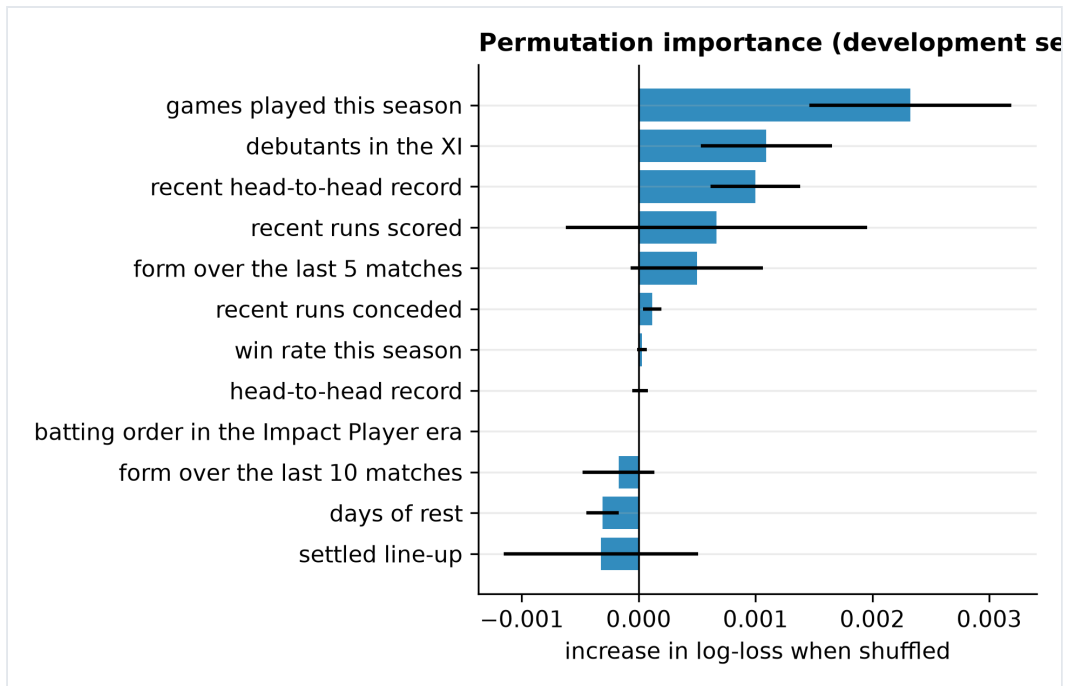


Figure 26: Permutation importance of the post-toss model (development seasons)

Table 20: Validation log-loss by era and stage (time-ordered calibrated out-of-fold predictions, post-toss)

Segment	n	Log-loss	Accuracy
era: impact_2023+	144	0.6955	52.1%
era: pre_2023	687	0.6847	53.3%
stage: league	779	0.6872	52.8%
stage: playoff	52	0.6776	57.7%

10.4 First-innings score regression

Table 21: First-innings runs, mean absolute error (MAE, runs)

Model	Walk-forward MAE	Holdout MAE [95% CI]	Holdout RMSE	Holdout bias
ridge	24.7	30.0 [26.1, 33.5]	37.1	-3.0
hist_gb	26.3	29.9 [26.4, 33.5]	37.0	-4.7
baseline_recent_league_mean	24.0	30.1 [26.3, 33.6]	37.3	-2.6
baseline_venue_level	25.1	35.6 [31.7, 39.6]	43.2	-22.4

A rolling league mean (the last ~74 matches) is a strong baseline because it adapts to the scoring jump in the Impact Player era. The venue-level baseline lags, with a large negative bias. The regressors add little beyond it, and the report says so.

10.5 Drift

The PSI compares the latest season with the development period. The largest shifts are:

- `venue_first_innings`: PSI 5.39 (alert)
- `h2h_meetings`: PSI 4.21 (alert)
- `bats_first_x_impact`: PSI 1.62 (alert)
- `bats_first_x_venue`: PSI 1.13 (alert)

- `elo_diff`: PSI 1.02 (alert)
- `venue_bat_first_rate`: PSI 0.63 (alert)

Scoring-level features moved far outside their training range after 2023. This explains the weaker holdout and would trigger retraining in production.

11. Testing Approach

11.1 Test pyramid

Table 22: Collected tests per suite

Suite	Tests	Content
Unit (tests/unit)	242	Every data, analytics, feature, model, service and reporting function; Hypothesis property tests; architecture (AST)
Integration (tests/integration)	26	Pipeline on isolated projects; CLI via <code>CliRunner</code> ; Streamlit <code>AppTest</code> for all 7 pages
Validation (tests/validation)	125	Leakage, reproducibility, experiment, security, documentation, performance
Total	393	

The integration and validation tests never write into the repository. They build **isolated temporary projects**: copies of the configs and external data with their own raw CSV, database and outputs.

11.2 Data-contract tests

- **Raw data:** the real CSV passes strict validation with the documented number of quarantined rows (0).
- **Corruption:** injected faults are caught with named reasons: bad toss decision, non-participant toss winner, 11 wickets on a non-tie, impossible date, same team twice, missing winner, short squad, non-constant overs.
- **Ground truth:** all 19 derived champions match published results; there are 74 playoff fixtures with era-correct labels; base rates are correct; every alias maps.

11.3 Leakage tests (written before any model code)

1. **Future deletion:** features for matches up to a cut-off are bitwise identical when all later matches are removed.
2. **Future mutation:** flipping every later result and inflating later scores changes no earlier feature.
3. **Same day:** removing one match of a double-header leaves the other's features unchanged.
4. **Label shuffle:** with shuffled labels, the model's mean AUC is within 0.03 of 0.5.
5. **Column allow-list:** post-match columns raise `FeatureLeakageError` .

In addition:

- The orientation flag alone has no signal (its bootstrap AUC CI contains 0.5).
- A documentation test shows that raw `team1` order *would* leak batting order.
- The "too good" guard fires above AUC 0.72.

11.4 Model and property tests

- **Hypothesis properties:**
- Elo expected scores are complementary and monotone.
- Updates are zero-sum and move in the right direction.
- The margin multiplier is positive and increasing.
- Shrunk rates lie between the raw rate and the prior.

- **Swap invariance:** $|p(A,B) - (1 - p(B,A))| < 1e-9$ for every model family, for calibrated predictions, and for served predictions.
- **Metrics:** cross-checked against scikit-learn and statsmodels (log-loss, Brier, AUC, Wilson, z-test, Holm).
- **Reproducibility:** two full experiment runs give identical selections, grids and holdout metrics (tolerance $1e-9$).
- **Registry:** round-trip loading works; an appended byte triggers `ModelIntegrityError`, and so does a missing artifact.

11.5 Security tests

- **Secrets scan** of every tracked file (GitHub tokens, AWS keys, private keys, key assignments). It guards against a regression of a real incident: a token pasted into the original build plan, which was stripped before the first commit.
- **SQL:** an AST ban on f-string or concatenated SQL in `text()`, `execute()` and `read_sql()`. Injection strings passed as team ids are inert.
- **Deserialisation:** allowed only in the registry, and only after the hash check. No `eval`, `exec` or `shell=True`.
- **Uploads:** wrong type, oversize, bad encoding, invalid rows and unknown teams are all rejected, with nothing written.
- **Exports:** CSV-injection payloads (`=`, `+`, `-`, `@`, tab) are escaped.
- **Static analysis:** `bandit` reports 0 issues and `pip-audit` finds no known vulnerabilities (`reports/bandit.json`, `reports/pip_audit.json`).

11.6 Coverage and continuous integration

Table 23: Line coverage (`coverage.py`)

data	features	models	analytics	total
98.7%	99.7%	98.9%	99.7%	94.6%

The GitHub Actions workflow runs a matrix of {Ubuntu, Windows, macOS} × {Python 3.12, 3.13}. Each run executes `ruff check`, `ruff format --check`, `mypy src`, `lint-imports` (the layer contract), `pytest --cov` and `bandit`, plus `pip-audit` once. All gates pass locally on Windows 11, and a clean clone (fresh virtual environment, `creaseiq all`, full test suite) was verified before release. CI runs on every push once the repository is published on GitHub.

12. Challenges Faced

These challenges actually occurred and were logged during the build (`PROGRESS.md` , ADRs).

1. **A column whose meaning changes over time.** From 2018 `team1` always bats first; before that the order is effectively random. The column's name does not reveal this, and it is easy to feed into a model. We discovered it by profiling the share of `team1` batting first per season. The fix was to derive batting order from the toss and orient matches with a hash-seeded coin flip (ADR-005), and a test now documents the trap.
2. **Super-over contamination.** The anomaly "12 wickets lost" turned out to be systematic. All 16 tie rows add super-over runs and wickets, and one adds two super overs. It was found only by checking scorecards during research (R8). The fix was verified regulation scores, with two independent sources per tie.
3. **A voided match that looks like a washout.** PBKS v DC on 08-05-2025 was stopped by a security blackout and replayed elsewhere. Without the research, it would have counted as a genuine no-result with scores.
4. **Low recall of the D/L heuristic.** Margin logic caught only the D/L results that contradict batting order. D/L chases "won by N wickets" are invisible in match-level data, so a researched external list was combined with the heuristic.
5. **Subtle venue identity.** Three kinds of trap appeared: a demolished and rebuilt stadium on the same site (Motera), a new ground in the same district (Mullanpur vs Mohali), and naming-rights renames (Sahara / MCA Pune). Research tallies also disagreed: the subagent's count said 40 venues, but the mapping itself yields 37, which we trust.
6. **A toolchain that was out of date.** The plan targeted Python 3.11, but the current numpy and scipy require 3.12, so the floor moved (ADR-002). The machine had no git or GitHub CLI, so both were installed before any version control could start.
7. **Secret hygiene.** A personal access token was pasted into the original build plan. It was removed before the first commit, authentication was delegated to `gh auth login` , and a secrets-scan test now guards the repository.
8. **Performance.** Under pandas 3, `itertuples` over about 950 small date groups made one feature build take 7 s, which made grid search and the leakage tests impractical. Profiling showed 117k indexer calls. Converting to plain records once cut the build to 0.33 s, a 21× speed-up.
9. **A negative holdout result.** On 2025–26 no model beat a coin flip. The temptation was to keep tuning until it did. Instead the selection was frozen and hashed before evaluation, and the result is reported as a finding about regime change. ADR-006 openly discloses one ledger reset after an exploratory run of the same selection.
10. **Plan errors found by review.** The plan's chase-rate baseline was computed over all seasons, which would have leaked the holdout base rate. Its toss test conditioned on a post-treatment choice. Both were corrected (`docs/plan_review.md` , 13 findings).
11. **Bugs caught by tests and types.** A logging handler wrote to a stream the test runner had closed; Streamlit's logging configuration silently disabled our module loggers; an output writer assumed `docs/` existed; mypy caught a shadowed loop variable in the registry code; and franchise alias validity windows ending in 2026 would have rejected any 2027 upload.
12. **A clean clone failed on Windows.** In the final QA run, a fresh `git clone` failed the raw-data SHA-256 check. Git for Windows (`core.autocrlf=true`) had rewritten the CSV's line endings on checkout. A `.gitattributes` file now marks data and model artifacts as byte-exact. The integrity check caught a problem that would otherwise have silently changed the input data.

13. **Unreliable research tools.** ESPNcricinfo blocks automated fetches, and the page summariser misreported some facts. The research therefore relied on raw Wikipedia wikitext and cross-checked sources, and every finding records its confidence.

13. Learnings & Key Takeaways

1. **Most of the work is data understanding.** The hardest and most valuable part was not the model. It was noticing that a column changes meaning in 2018, that ties hide super overs, and that one "no result" was a voided match. Profiling, domain research and fail-loud validation are worth more than an extra model family.
2. **Leakage is easy to create and hard to see.** Aggregates computed over the whole dataset, random splits, post-match columns and even row orientation can all leak. The reliable defence is structural (as-of date batches, allow-lists) backed by adversarial tests that *try* to leak.
3. **Honest evaluation beats impressive numbers.** Walk-forward validation, baselines, confidence intervals and a locked holdout showed that pre-match IPL prediction is barely better than a coin flip. The literature agrees (T20 is highly random [2], [3]). Claims of 80–90% accuracy are a warning sign, not a target.
4. **Probabilities matter more than labels.** Log-loss and calibration measure whether "60%" really means 60%. Accuracy can look fine while the probabilities are useless, and selecting on calibration is the more useful practice [9].
5. **Statistical tests need effect sizes and power.** "Not significant" is only informative with a minimum detectable effect. The toss analysis rules out effects larger than about four points; it does not "prove zero". Separating causal from associational questions (the toss vs the decision) prevents wrong conclusions.
6. **Symmetry can be designed in.** Declaring features antisymmetric and dropping the intercept made the logistic model exactly invariant to team order. Tests then verify the invariance mechanically.
7. **Simplicity wins on small data.** The one-SE rule repeatedly chose Elo-logit or logistic models over forests and boosting. With about 1k samples, variance dominates.
8. **Engineering discipline makes research reproducible.** Seeds, pinned dependencies, config files, a registry with hashes, idempotent rebuilds, a holdout ledger and CI turn "it worked on my laptop" into evidence.
9. **Layering pays for itself.** Because the dashboard and the CLI share the same services, every feature was testable once, without a browser.
10. **Documentation is part of the product.** ADRs recorded *why* decisions were made. That made the plan review, the report and viva preparation straightforward.

14. Future Enhancements

1. **Ball-by-ball data.** Ingest Cricsheet ball-by-ball JSON through the existing `MatchSource` interface. This would enable in-play win probability (given the score, wickets and balls left), player batting and bowling ratings, and phase-wise analytics (powerplay and death overs).
2. **Player-level team strength.** Build pre-match squad strength from player ratings and auction data. Squad turnover is the main reason history-only features fail after mega-auctions.
3. **Regime-aware models.** Down-weight older seasons (time-decayed training), add explicit era terms, or use Bayesian hierarchical models with team-by-season effects. All of these would be evaluated with the same walk-forward protocol.
4. **Conditions data.** Add weather and dew (evening humidity), pitch reports and venue-by-era effects, which are the known confounders of toss and chasing results.
5. **Automated drift response.** Trigger retraining when the PSI crosses its alert threshold, and use champion/challenger evaluation before promoting a model.
6. **A service interface.** Expose a REST API (e.g. FastAPI) over the existing services layer, with authentication and rate limiting for multi-user use.
7. **Deployment.** Package in a container and deploy the read-only dashboard to Streamlit Community Cloud with PostgreSQL. The database URL switch already exists.
8. **Data pipeline hardening.** Add a scheduled incremental ingest of new fixtures, with the same strict validation and dry-run canonicalisation used for uploads.
9. **Better player identity.** Adopt Cricsheet registry person ids instead of exact name strings, which would remove the known ambiguous name.
10. **Accessibility.** Add a full keyboard and screen-reader audit of the dashboard, a high-contrast theme and text alternatives for every chart.

15. References

All sources were accessed on 28 September 2026 and actually opened during the research phase (docs/research_notes.md). The ESPNcricinfo and journal pages that blocked automated access are cited only where an accessible version was read. Link check performed at report build time: 14 of 14 URLs resolved.

- [1] S. Rushe, "Cricsheet: IPL match data (JSON) and Register," cricsheet.org. Available: <https://cricsheet.org/matches/> and <https://cricsheet.org/register/> (Open Data Commons Attribution License v1.0).
- [2] G. Sood and D. Willis, "Fairly random: The impact of winning the toss on the probability of winning," arXiv:1605.08753, 2016.
- [3] S. Kampakis and W. Thomas, "Using machine learning to predict the outcome of English county twenty over cricket matches," arXiv:1511.05837, 2015.
- [4] N. Lamsal and A. Choudhary, "Predicting outcome of Indian Premier League (IPL) matches using machine learning," arXiv:1809.09813, 2018.
- [5] L. M. Hvattum and H. Arntzen, "Using ELO ratings for match result prediction in association football," *International Journal of Forecasting*, vol. 26, no. 3, pp. 460–470, 2010.
- [6] FiveThirtyEight, "nfl-elo-game: forecast.py," GitHub repository. Available: <https://github.com/fivethirtyeight/nfl-elo-game>
- [7] N. Johnsson, "A simple improvement to FiveThirtyEight's NBA Elo model," Harvard Sports Analysis Collective, 2019.
- [8] P. Puram, S. Roy, D. Srivastav and S. Gurumurthy, "Contextual factors and decision making in T20 cricket: an interpretable machine learning approach," *Annals of Operations Research*, vol. 325, 2023.
- [9] C. Walsh and A. Joshi, "Machine learning for sports betting: should model selection be based on accuracy or calibration?," *Machine Learning with Applications*, 2024 (arXiv:2303.06021).
- [10] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.
- [11] V. Cerqueira, L. Torgo and I. Mozetič, "Evaluating time series forecasting models: an empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020.
- [12] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd ICML*, 2005.
- [13] scikit-learn developers, "Probability calibration," scikit-learn User Guide. Available: <https://scikit-learn.org/stable/modules/calibration.html>
- [14] J. Nixon, M. Dusenberry, L. Zhang, G. Jerfel and D. Tran, "Measuring calibration in deep learning," in *CVPR Workshops*, 2019 (arXiv:1904.01685).
- [15] A. Kumar, P. Liang and T. Ma, "Verified uncertainty calibration," in *NeurIPS*, 2019 (arXiv:1909.10155).
- [16] R. Roelofs, N. Cain, J. Shlens and M. C. Mozer, "Mitigating bias in calibration error estimation," in *AISTATS*, 2022 (arXiv:2012.08668).
- [17] "Brier score" (Murphy decomposition), Wikipedia. Available: https://en.wikipedia.org/wiki/Brier_score
- [18] Real Statistics Using Excel, "Diebold-Mariano test." Available: <https://real-statistics.com/time-series-analysis/forecasting-accuracy/diebold-mariano-test/>

- [19] S. Bhaskar, "Rational adversaries? Evidence from randomised trials in one day cricket," *The Economic Journal*, vol. 119, no. 534, pp. 1–23, 2009.
- [20] "List of Indian Premier League seasons and results," "2025 Indian Premier League," "2026 Indian Premier League," Wikipedia. Available: https://en.wikipedia.org/wiki/List_of_Indian_Premier_League_seasons_and_results
- [21] Open Data Commons, "Attribution License (ODC-By) v1.0 summary." Available: <https://opendatacommons.org/licenses/by/summary/>
- [22] "Promotion and Regulation of Online Gaming Act, 2025," Wikipedia. Available: https://en.wikipedia.org/wiki/Promotion_and_Regulation_of_Online_Gaming_Act,_2025
- [23] Python Software Foundation, "Status of Python versions," Python Developer's Guide. Available: <https://devguide.python.org/versions/>
- [24] Streamlit, "st.testing.v1.AppTest," Streamlit documentation. Available: <https://docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest>
- [25] Microsoft, "Page.pdf," Playwright for Python documentation. Available: <https://playwright.dev/python/docs/api/class-page#page-pdf>
- [26] Cricsheet, "JSON format specification." Available: <https://cricsheet.org/format/json/>
- [27] Vellore Institute of Technology, "Research Integrity Policy." Available: <https://vit.ac.in/files/ebooks/Research-Integrity-Policy/files/basic-html/page7.html>

Appendix: AI assistance statement

This project was built with AI coding assistance (Claude Code), used for scaffolding, literature and fact research, and review. The author reviewed and approved the design decisions (ADR-001...006) and can explain every module; see `docs/viva_prep.md`. Each external fact carries its source and confidence in `docs/research_notes.md`. No code was copied from the IPL projects surveyed for differentiation.